DSAA 4011 · LECTURE 10 · THURSDAY 12 NOVEMBER 2026

Amortized Inference

One network, trained once, replaces the optimisation we have been running for every new observation

Builds on Lecture 5 (expectation–maximization) and Lecture 6 (the **evidence lower bound**)

Yuwen Huang
Advanced Machine Learning and Deep Learning · HKUST (Guangzhou)

OPENING

One story, then one example, then applications

We will understand the method before calculating with it.

Pass	Question
I · Theory	What do we infer, optimise and approximate?
II · Example	How do all these ideas work in one fixed model?
III · Applications	Why are these ideas useful in larger systems?

SO **The same concepts return in each pass.**

NEXT Why replace inference by a learned function?

CLASSROOM CHECKPOINT · PREDICT, EXPLAIN, THEN REVEAL

By the end, explain it and perform one update

We will revisit the same goals after the theory and after the numerical example.

You should be able to	Evidence of learning
Explain why an encoder is useful	Describe the repeated inference it replaces
Explain what its training score rewards	Distinguish $\log p(x, z) - \log q_\varphi(z \mid x)$ from $\log p(x, z)$
Perform one parameter update	Name the inputs and calculate each step
Diagnose an imperfect latent representation	Distinguish the two gaps and collapse

SO **We will check understanding, not just display formulas.**

NEXT How will we use the three hours?

CLASSROOM CHECKPOINT · PREDICT, EXPLAIN, THEN REVEAL

Three hours include practice, questions and a break

The main lecture follows three passes. The supplementary pages are for optional reference.

Activity	Minutes
Opening and prior-knowledge check	10
I · Theory, with prediction questions	45
Break	10
II · One example, with student calculations	65
III · Applications and interpretation	25
Retrieval questions and explanation	15
Questions and flexible time	10

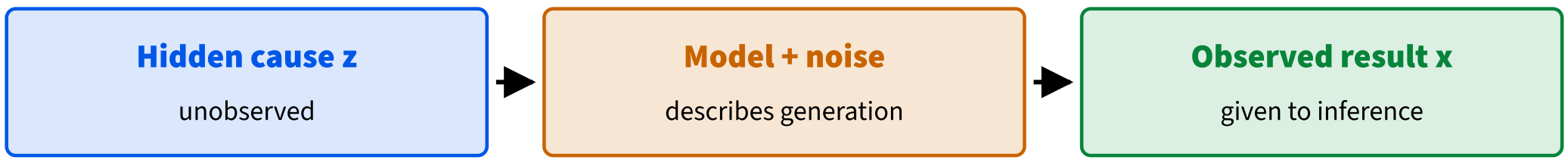
SO **Understanding determines the pace; optional derivations can wait.**

NEXT What is observed, and what must inference recover?

Many observations repeat the same inference task

A generative model describes how a hidden cause produces an observation.

Known	Wanted
Observation x	A distribution over its hidden cause
Model $p(z), p(x \mid z)$	Posterior $p(z \mid x)$



Generation runs left to right; inference asks about the hidden cause.

SO **Each new observation creates another inference problem.**

NEXT Which earlier results already address this inference task?

The task is familiar; who computes the guess changes

Earlier lectures already supplied the posterior target, the lower bound and the idea of learned inference.

Earlier result	Its role today
Lecture 3: Kullback–Leibler (KL) divergence is nonnegative	Explains why $\mathcal{L} \leq \log p(x)$
Lecture 5: expectation–maximization (EM) infers hidden variables, then updates the model	We replace separate inference calculations with a shared encoder
Lecture 6: optimise a variational distribution	Keep the same lower bound; change how the guess is parameterised
Lecture 9: learn a reusable inference rule	Apply the same idea to a posterior-distribution encoder

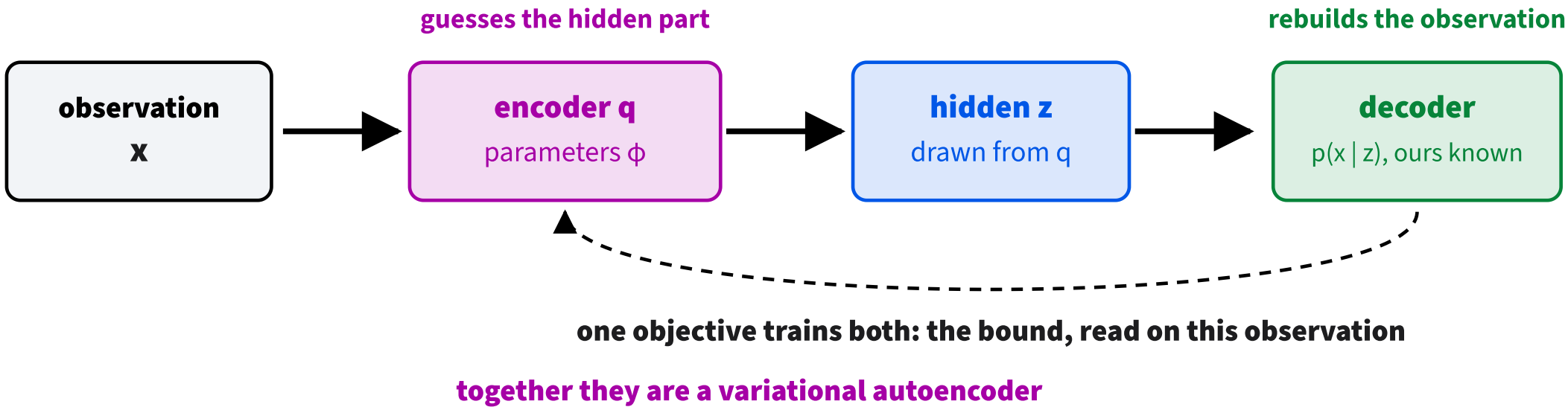
SO **Today's new ingredient is shared inference parameters, not a new posterior target.**

NEXT What does this shared inference function output?

One encoder proposes a distribution for each input

Rather than optimise a new guess from scratch, learn a reusable function.

$$x \longrightarrow q_\varphi(z \mid x)$$



SO φ **are shared parameters; the output is a distribution, not one hidden answer.**

NEXT What can train this function when exact posteriors are unavailable?

Three expressions, three different roles

Keep the model fixed. Each term below answers a different question.

Expression	Meaning
$\log p(x, z)$	Log-joint contribution only; not the full training term
$\log p(x, z) - \log q_\varphi(z \mid x)$	Full sample training term: joint minus encoder
$\mathcal{L}(\varphi; x) = \mathbb{E}_q[\log p(x, z) - \log q_\varphi(z \mid x)]$	Expected training score: the evidence lower bound

SO **One candidate's score and the expected score $\mathcal{L}$ are different quantities.**

NEXT Why can we evaluate or estimate these expressions?

A computable score trains the posterior guesses

For each proposed hidden value, compare the model and encoder probabilities.

$$\mathcal{L}(\varphi; x) = \mathbb{E}_{z \sim q_\varphi}[\underbrace{\log p(x, z) - \log q_\varphi(z \mid x)}_{\text{one full sample training term}}]$$

Assume the generative model is already given: $p(z)$ and $p(x \mid z)$ are known and fixed. Observe x, then let the encoder propose z. Evaluate $p(x, z) = p(z)p(x \mid z)$ to score that proposal. **Here we train only the encoder.**

Why computable? Draw z from the encoder; evaluate the known $p(x, z)$ and $q_\varphi(z \mid x)$; average the scores. **No true posterior needed.** Sampling estimates the expectation.

Known model	**Observed input x**	**Encoder proposal z**
evaluate p(x,z)	keep fixed	evaluate q(z | x)

Use both probabilities to score one candidate, then average.

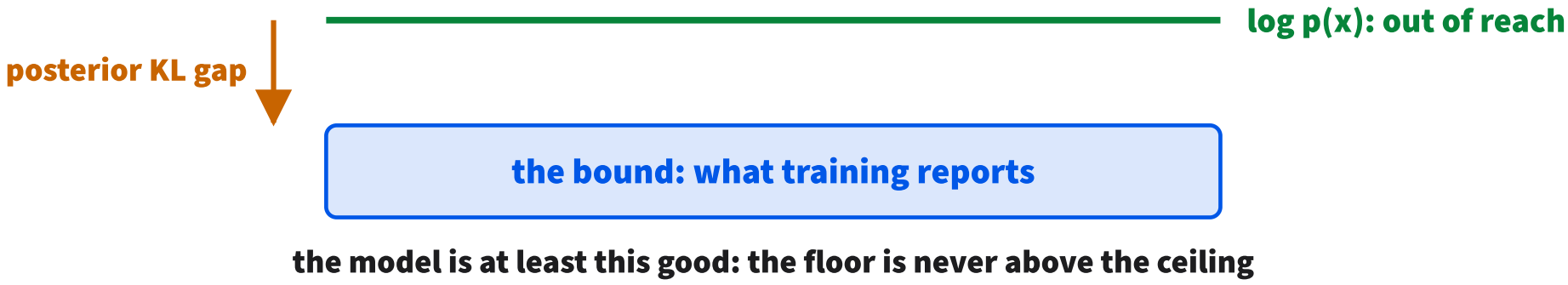
SO **Raise the average score by changing the encoder, without posterior labels.**

NEXT Why should a higher score improve the guess?

The score is evidence minus posterior error

Hold the generative model fixed while comparing encoder guesses.

$$\log p(x) = \mathcal{L}(\varphi; x) + \underbrace{\mathrm{KL}(q_\varphi \| p(z \mid x))}_{\text{nonnegative posterior gap}}$$



SO **Higher bound means smaller posterior divergence; equality requires an exact posterior guess.**

NEXT How do we train one encoder using many observations?

All observations update the same encoder

Each observation contributes its own lower bound.

$$\text{training objective} = \sum_k \mathcal{L}(\varphi; x_k)$$

Observation 1

score L1

Observation 2

score L2

Observation 3

score L3

All scores update the same encoder parameters.

SO x_k **is observation** k; **every term updates the same** φ.

NEXT What if the encoder cannot represent all the right answers?

Two restrictions leave a posterior KL gap

First ask what each best separate guess can express; then impose a shared function.

Restriction	Name of the resulting loss
Allowed distributions cannot match a posterior	Approximation gap
Shared encoder cannot reach the best separate guesses	Amortization gap



These are different sources of error; one remedy need not fix all three.

SO **Incomplete optimisation can add further loss beyond these best attainable gaps.**

NEXT If $\log p(x, z) - \log q_\varphi(z \mid x)$ contains a difficult expectation, how do we evaluate it?

CLASSROOM CHECKPOINT · PREDICT, EXPLAIN, THEN REVEAL

Can longer training repair a missing input?

Discuss with a neighbour for one minute. Give a reason, not only an answer.

Two different observations become the same encoder input. Can more training produce two different outputs?

No: the inputs are identical | Yes: train longer | Yes: add more layers

Answer. No. The same deterministic encoder input gives the same output; restore the missing information.

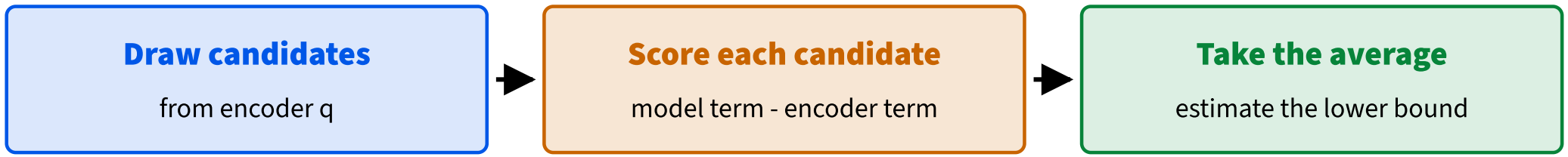
SO **Identify the restriction before changing the optimiser.**

NEXT If the expected score is hard to compute, how can samples estimate it?

Average sampled scores when the integral is hard

Draw possible hidden values from the encoder and score them using the same rule.

Operation	Meaning
Draw	Generate candidates from the current guess
Score	Compute model log probability minus encoder log probability
Average	Estimate the expected score



More independent draws can make the estimated average more reliable.

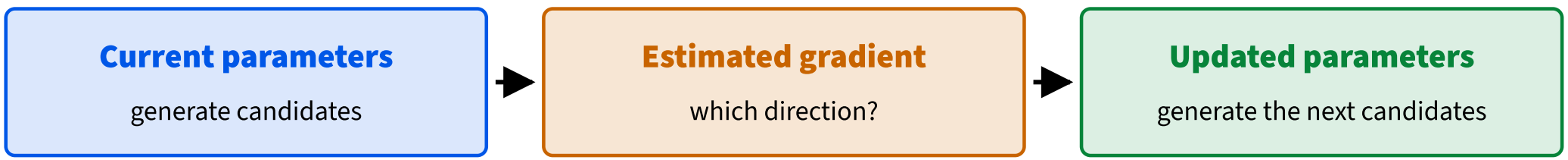
SO **Sampling approximates the lower bound $\mathcal{L}$; it does not reveal the true hidden value.**

NEXT How do the sampled scores tell us which parameters to change?

Gradients tell the encoder which way to change

We want the sampling rule to generate candidates with higher average training scores.

At the current parameters	Small gradient-ascent update
Positive derivative	Increase that parameter
Negative derivative	Decrease that parameter



Training repeats this loop; a random draw is not itself a parameter.

SO **Improve the distribution's expected score, rather than guaranteeing every draw improves.**

NEXT How do we differentiate a score whose samples also depend on the encoder?

Build a random sample in two simple steps

The encoder chooses where the guess is centred and how spread out it is.

Step	Action
1 · Draw a random offset	Draw ε from the fixed standard normal $N(0, 1)$
2 · Place the candidate	Multiply by the encoder's spread s, then add its centre μ

$$\underbrace{z}_{\text{candidate}} = \underbrace{\mu}_{\text{centre}} + \underbrace{s}_{\text{standard deviation}} \cdot \underbrace{\varepsilon}_{\text{random offset}}$$

To calculate **one gradient**, keep that offset fixed and track how changing the encoder moves the candidate. Draw fresh offsets for subsequent estimates.

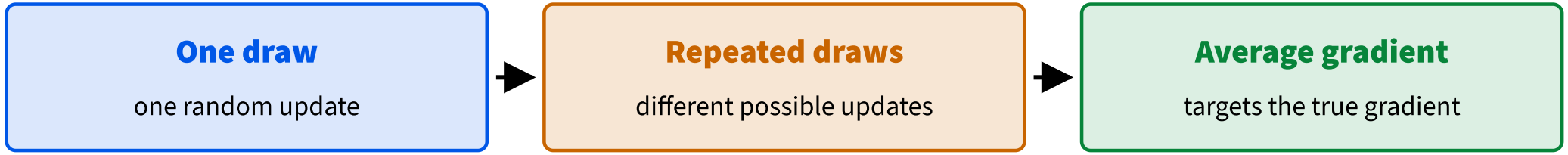
SO **The encoder controls centre and spread; randomness comes from a separate, unchanged noise distribution.**

NEXT Why can these random gradient estimates be useful on average?

A correct average can still yield noisy updates

Under regularity conditions, sampled gradients average to the true gradient.

What follows	What does not follow
The estimator is unbiased	Every step improves the objective
Averaging can reduce sampling noise	Training reaches a global optimum



A correct average does not make every individual step correct.

SO **Convergence needs additional conditions; unbiasedness alone is insufficient.**

NEXT Can even an optimised encoder leave the hidden variable unused?

CLASSROOM CHECKPOINT · PREDICT, EXPLAIN, THEN REVEAL

What is the encoder actually being trained to do?

Take one minute to explain the goal without using the word “gradient”.

Which objective have we defined?

Raise the full average score | Raise every sample’s log joint | Recover one hidden value exactly

Answer. Raise the expected log joint minus encoder log density; this targets a distribution, not one best hidden value.

SO **Training improves an expected score; individual random draws can be worse.**

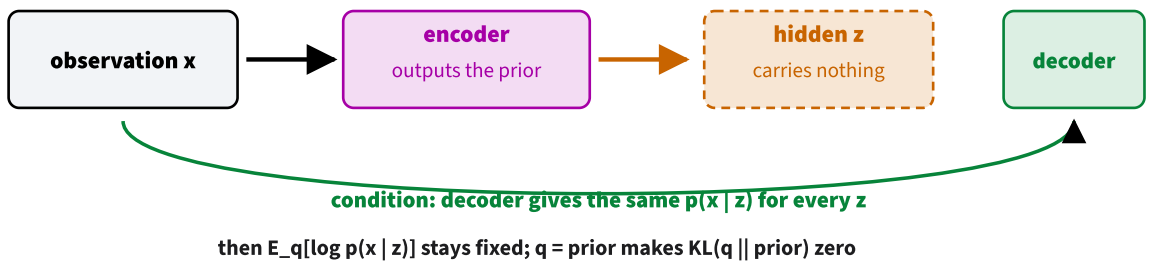
NEXT What happens if the decoder does not need the hidden variable?

A hidden variable can become unnecessary

Read the same bound as fit to the observation minus departure from the prior.

$$\mathcal{L} = \underbrace{\mathbb{E}_q \log p(x \mid z)}_{\text{fit}} - \underbrace{\mathrm{KL}(q(z \mid x) \| p(z))}_{\text{prior penalty}}$$

Fit: do the proposed hidden values make the observed x likely?
Prior penalty: how far does the whole guessed distribution $q(z \mid x)$ move from our original assumption $p(z)$? It is zero when they match.



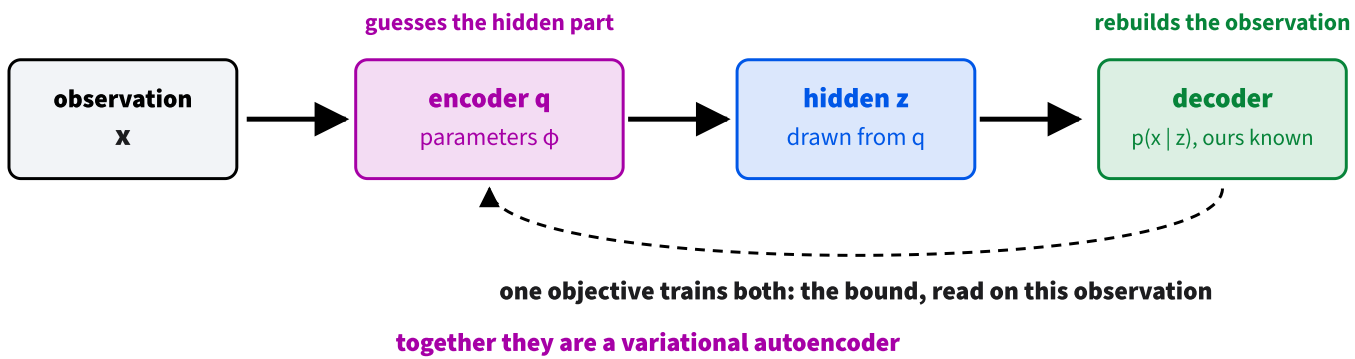
SO **If the decoder gives the same observation probabilities for every z, changing the encoder cannot change fit. Choosing q = prior then removes the prior penalty $\mathrm{KL}(q(z \mid x) \| p(z))$. This is a condition, not a consequence of network size.**

NEXT What changes when the generative model also learns?

Learning the model adds a decoder to the story

The encoder learns hidden-value guesses; the decoder learns how hidden values explain observations.

Parameters	Role
Encoder φ	Proposes a distribution from an observation
Decoder θ	Specifies observation probabilities from a hidden value



SO **the lower bound $\mathcal{L}$ can rise because the encoder reduces its posterior error, even if the changing model gives the data lower likelihood.**

NEXT How does learning the encoder and decoder relate to the E-step and M-step?

The same lower bound connects EM and the encoder

Recall $\mathcal{L}(\theta, q; x) = \mathbb{E}_q[\log p_\theta(x, z) - \log q(z)]$. θ controls the model; q guesses the hidden variable.

Method	Infer the hidden variable	Learn the model
Exact expectation–maximization (EM) · Lecture 5	E-step: set each q to its exact posterior	M-step: maximise the bound in θ
Variational EM · Lecture 6	Optimise each q within an allowed family	Improve the same bound in θ
Amortized inference · Today	One shared $q_\varphi(z \mid x)$ supplies the guesses	When learned, update decoder θ too

Different guarantees: exact EM's E-step makes the bound tight before its M-step. A shared approximate encoder need not do so; joint gradient updates do not inherit EM's monotonic likelihood guarantee.

SO **The encoder shares the work of the E-step; it is not automatically an exact E-step.**

NEXT What exact E-step calculation does an encoder replace in our Gaussian example?

The encoder replaces per-observation posterior solving

Keep $z \sim N(0,1), x = z + \eta, \eta \sim N(0,1)$, and observed $x = 1.6$. Hold the model fixed.

Exact expectation–maximization (EM) E-step	Trained encoder
Compute $p(z \mid x) = N(x/2, 1/2)$	Evaluate $q_\varphi(z \mid x) = N(ax, s^2)$
For this observation: $N(0.8, 0.5)$	Learned $a = 0.5$, $s^2 = 0.5$: output $N(0.8, 0.5)$
Repeat posterior inference for each observation	Reuse the same learned mapping on each observation

Why faster in difficult models? Replace many inference iterations per observation with one network evaluation. Pay for training beforehand. **This Gaussian E-step already has a cheap formula, so it does not demonstrate a speedup.**

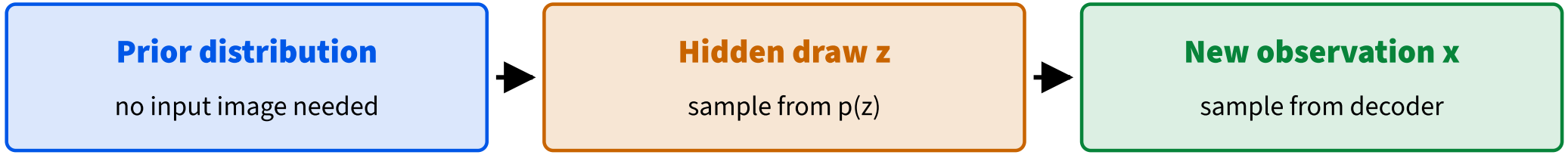
SO **The encoder replaces how the E-step's posterior guess is obtained, not the model update. If the model changes, the encoder may need updating too.**

NEXT How are these components used for reconstruction and fresh generation?

Training uses the encoder; generation starts from the prior

The encoder explains existing observations. The generative model can also create new ones.

Task	Route
Reconstruct an observation	Given x → draw $z \sim q_\varphi(z \mid x)$ → decode
Generate a new observation	Draw $z \sim p(z)$ → draw $x \sim p_\theta(x \mid z)$
Train on data	Encode → sample → score → update both networks



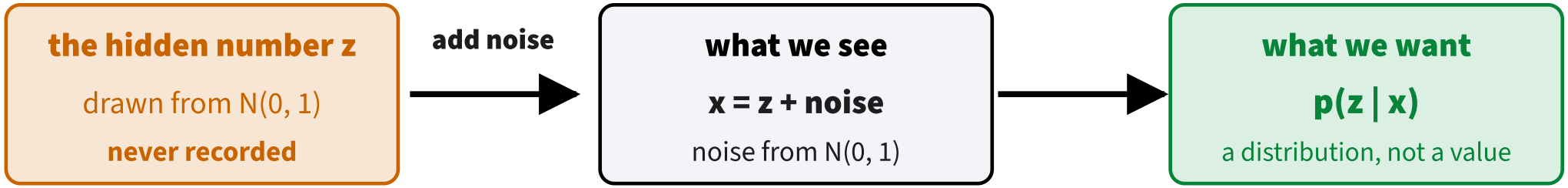
This is generation; encoding is needed when explaining an existing observation.

SO **Generating a fresh observation does not require encoding an existing one.**

NEXT Can we now compute the method in one fixed model?

Infer a hidden number from a noisy observation

Use one model throughout: $z \sim N(0,1), x = z + \eta$, with independent $\eta \sim N(0,1)$. Observe $x = 1.6$.



infer the hidden number: the encoder outputs a mean and a standard deviation

SO **We observe the noisy value and infer a distribution for the hidden number.**

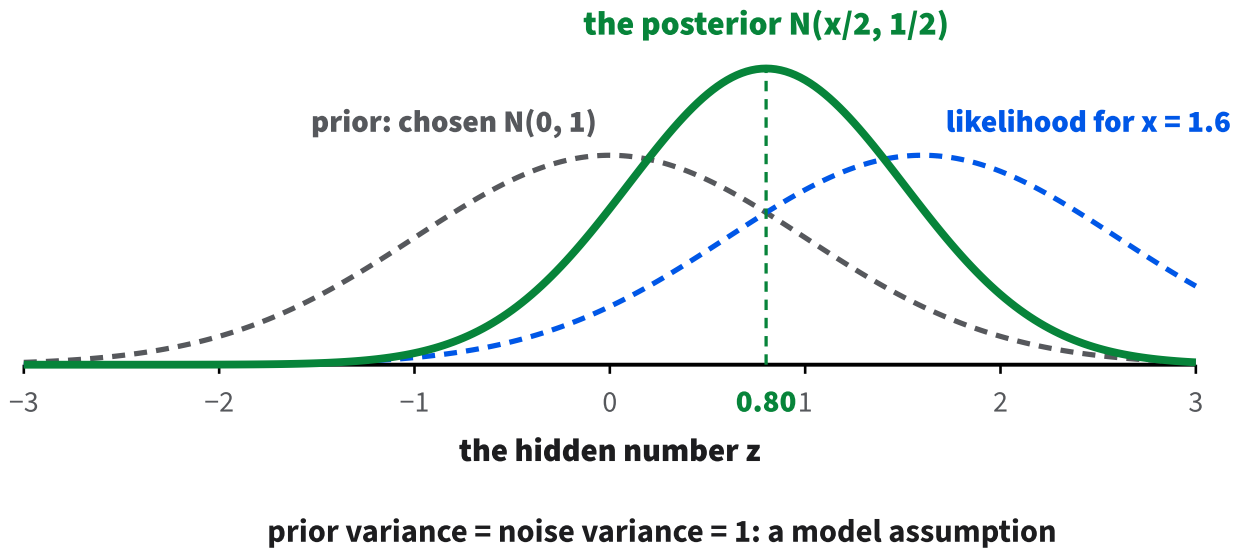
NEXT How does the observation determine the posterior in this model?

The observation shifts our guess toward its value

Combine the chosen **prior** with the information from one noisy observation.

Our chosen model: $z \sim N(0,1); x = z + \eta$, **with independent noise** $\eta \sim N(0,1)$. **Here** $N(\text{mean}, \text{variance})$.

Observe $x = 1.6$. The **prior** centres on 0; the likelihood favours z near 1.6, where the required **noise** $1.6 - z$ is near zero.



prior variance = noise variance = 1: a model assumption

SO **Equal prior and noise variances place the posterior mean halfway.**

NEXT How does the observation determine the posterior in this model?

Multiply the two curves, then complete one square

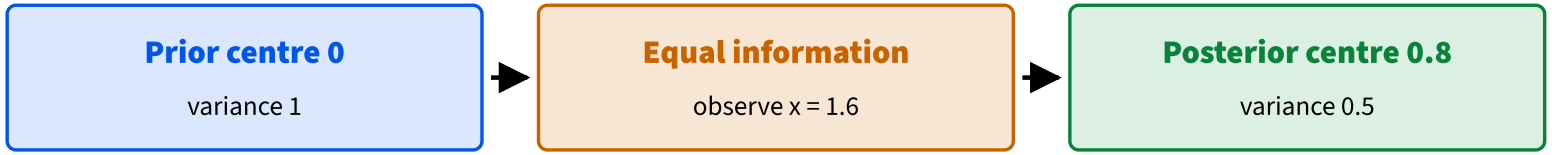
The green curve comes from multiplying the prior and likelihood.

$$p(z \mid x) \propto e^{-z^2/2} \cdot e^{-(x-z)^2/2} = e^{-\{z^2+(x-z)^2\}/2}$$

Rearrange. $z^2 + (x-z)^2 = 2z^2 - 2xz + x^2 = 2(z - x/2)^2 + x^2/2.$

Read the bell. The term $x^2/2$ is constant in z, so normalization absorbs it: $p(z \mid x) \propto e^{-(z-x/2)^2} = e^{-(z-\mu)^2/(2v)}$. Thus $\mu = x/2, v = 1/2$.

For our observation. Mean: $1.6/2 = \mathbf{0.8}$. **Variance**: $\mathbf{0.5}$. Equal **prior** and noise variances give an equal-weight mean; **two clues reduce uncertainty**.



The product favours hidden values supported by both sources.

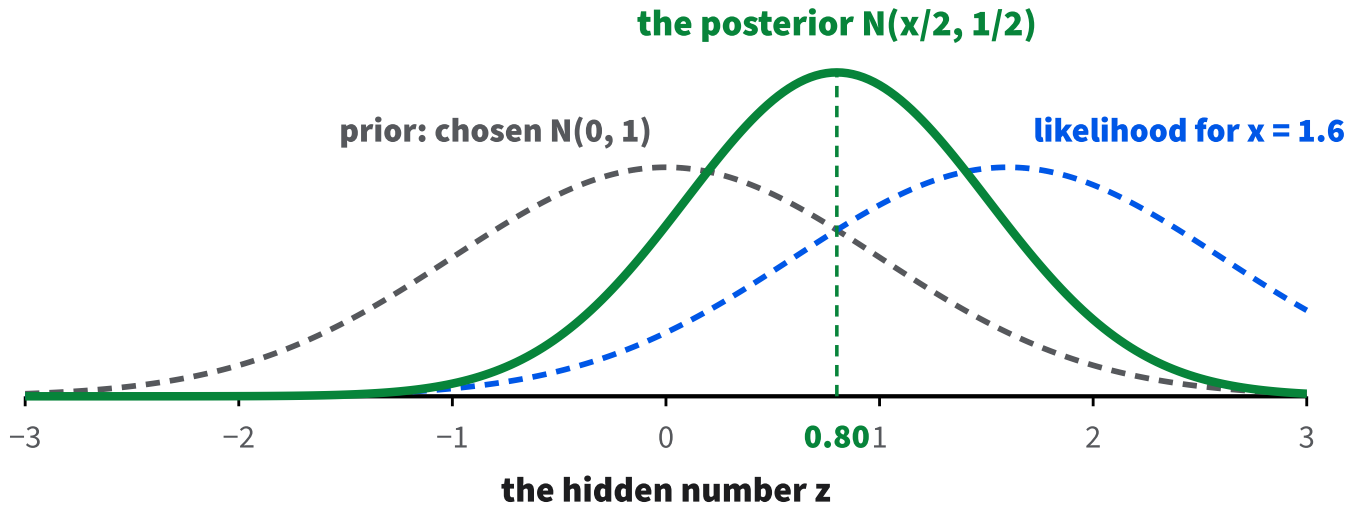
SO **Completing the square gives posterior mean x/2 and variance one half.**

NEXT What distribution will the encoder learn to output?

The encoder learns a centre and a spread

Our Gaussian encoder needs a rule for its mean and a positive spread.

> Encoder: $q(z \mid x) = N(a \cdot x, s^2)$. Learn a (mean slope) and $s > 0$ (standard deviation).



prior variance = noise variance = 1: a model assumption

SO **Two shared parameters can express this model's posterior for every observation.**

NEXT How does the lower bound $\mathcal{L}$ score our starting guess and the exact posterior?

A better guess gains fit while paying a prior penalty

Keep $x = 1.6$. Compare the starting guess with the exact posterior; both use the same model.

Fit: can our guessed z explain the observation $x = 1.6$?
Prior penalty: how far is the guessed distribution from the prior $N(0, 1)$? Training maximizes **fit minus penalty**.

Guess	Expected log likelihood	Prior penalty	Bound = fit − penalty
Start: $N(0, 1)$	−2.699	0	−2.699
Posterior: $N(0.8, 0.5)$	−1.489	0.417	−1.906

Fit: $-\frac{1}{2}\log(2\pi) - \frac{1}{2}[(1.6 - \mu)^2 + s^2]$. Prior penalty: $\frac{1}{2}[\mu^2 + s^2 - 1 - \log s^2]$. Here μ is the guess mean; s^2 is its variance.

SO **The posterior earns more fit than it pays in extra penalty. A zero prior penalty alone is not the goal.**

NEXT How do gradients find a better guess without posterior labels?

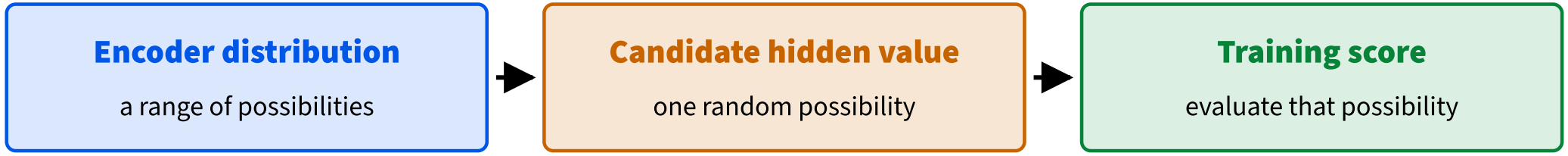
A sample is one possible answer, not the true answer

An encoder proposes possible hidden values; it does not reveal the true hidden answer.

Encoder's guess	Sampled candidates
A bell centred at $\mu = 0$	For illustration: $-1, 0, 1$

Draw a few possible values from the encoder. Score each one using the **same rule as before**: $\log p(x, z) - \log q(z \mid x)$.

Take their **average**: for example, scores $-2, -1, -3$ average to -2. These scores are illustrative.



Repeat the draw and average scores; no candidate is a known true answer.

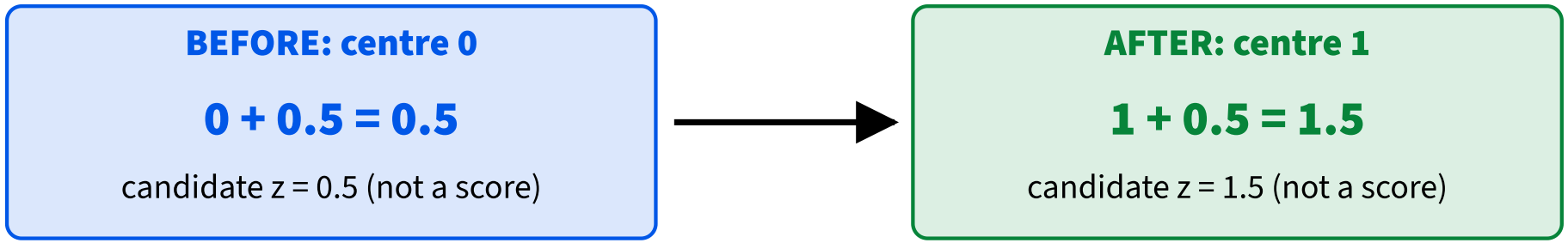
SO **The average sample score estimates the lower bound we want to raise.**

NEXT How can we compare candidates before and after changing the guess?

Move the guessed centre; follow one candidate

We want to test a change to the encoder's guess. Keep the observation and the bell's spread unchanged.

Illustration: use the same random offset +0.5



Change: centre Keep: observation, spread, random offset

Only candidate values are shown here; no training scores have been calculated.

Use the **same random offset** in both cases. Then the candidate changes only because we changed the centre.

Candidate value is not its score. After generating z, compute $\log p(x, z) - \log q_\varphi(z \mid x)$. Its average is $\mathcal{L}$, equivalently expected fit minus prior penalty.

SO **Changing the centre changes the candidate we score. Its score may rise or fall.**

NEXT Let us calculate the actual scores of candidates 0.5 and 1.5.

First specify the model and both encoder guesses

Keep $x = 1.6$, prior $p(z) = N(0, 1)$, and likelihood $p(x \mid z) = N(z, 1)$. Only the encoder centre changes.

Encoder	Same noise offset	Generated candidate
Before: $q_0(z \mid x) = N(0, 1)$	$s = 1, \varepsilon = 0.5$	$z = 0 + 1 \cdot 0.5 = 0.5$
After: $q_1(z \mid x) = N(1, 1)$	$s = 1, \varepsilon = 0.5$	$z = 1 + 1 \cdot 0.5 = 1.5$

For a unit-variance normal: $\log N(t; m, 1) = \underbrace{-\tfrac{1}{2}\log(2\pi)}_{c_N \approx -0.91894} - \tfrac{1}{2}(t - m)^2$.

This follows by taking logs of $N(t; m, 1) = \frac{1}{\sqrt{2\pi}} e^{-(t-m)^2/2}$. Use the same formula for all three density terms.

SO **Candidate values have been generated; neither number is a training score.**

NEXT What score does candidate 0.5 receive under the first encoder?

Candidate z = 0.5 receives score about −1.524

Before: $q_0 = N(0, 1)$, **candidate** $z = 0.5$, **observed** $x = 1.6$. **Use** $c_N \approx -0.91894$.

Term	Substitute the values	Result
Prior $\log p(z)$	$c_N - \frac{1}{2}(0.5 - 0)^2 = c_N - 0.125$	−1.04394
Likelihood $\log p(x \mid z)$	$c_N - \frac{1}{2}(1.6 - 0.5)^2 = c_N - 0.605$	−1.52394
Joint $\log p(x, z)$	Add the first two rows: −1.04394 − 1.52394	−2.56788
Encoder $\log q_0(z \mid x)$	$c_N - \frac{1}{2}(0.5 - 0)^2 = c_N - 0.125$	−1.04394
Full sample training term	Joint − encoder: −2.56788 − (−1.04394)	−1.52394

SO **Candidate: 0.5. Its score: approximately −1.524. Subtracting a negative encoder log density adds its magnitude.**

NEXT Now compute the score of candidate 1.5 under the second encoder.

Candidate z = 1.5 receives score about −1.924

After: $q_1 = N(1, 1)$, candidate $z = 1.5$. Keep $x = 1.6$ and the generative model unchanged.

Term	Substitute the values	Result
Prior $\log p(z)$	$c_N - \frac{1}{2}(1.5 - 0)^2 = c_N - 1.125$	−2.04394
Likelihood $\log p(x \mid z)$	$c_N - \frac{1}{2}(1.6 - 1.5)^2 = c_N - 0.005$	−0.92394
Joint $\log p(x, z)$	Add the first two rows: −2.04394 − 0.92394	−2.96788
Encoder $\log q_1(z \mid x)$	$c_N - \frac{1}{2}(1.5 - 1)^2 = c_N - 0.125$	−1.04394
Full sample training term	Joint − encoder: −2.96788 − (−1.04394)	−1.92394

SO **Candidate: 1.5. Its score: approximately −1.924. A larger candidate value does not mean a better score.**

NEXT How does this full sample training term relate to fit minus prior penalty?

A sample contribution is not the expected objective

The first candidate scored higher: $-1.524 > -1.924$. This compares two individual draws, not the two encoders' expected objectives.

One draw: $\underbrace{\log p(x \mid z)}_{\text{likelihood at the candidate}} + \underbrace{\log p(z) - \log q_\varphi(z \mid x)}_{\text{prior versus encoder at the candidate}}$.

Average over $z \sim q_\varphi$: $\mathcal{L} = \underbrace{\mathbb{E}_q \log p(x \mid z)}_{\text{expected fit}} - \underbrace{\mathrm{KL}(q_\varphi(z \mid x) \| p(z))}_{\text{prior penalty for the whole distribution}}$.

Why the minus sign? $\mathrm{KL}(q\|p) = \mathbb{E}_q[\log q - \log p]$. The average of $\log p - \log q$ is therefore its negative.

Not the prior penalty by itself: $-\log q(z \mid x)$ is the encoder correction at one sample. The prior penalty is $\mathrm{KL}(q\|p(z)) = \mathbb{E}_q[\log q - \log p(z)]$, which includes the prior term too.

SO **Train the expected objective $\mathcal{L} = \mathbb{E}_q[\log p(x, z) - \log q_\varphi(z \mid x)]$, not the log joint alone.**

NEXT How do we write the same-noise sampling rule for a general Gaussian encoder?

Reuse one random offset when moving the centre

A Gaussian candidate is its centre plus a random offset. Keep the standard deviation at one for this example.

$$\underbrace{z}_{\text{candidate}} = \underbrace{\mu}_{\text{centre}} + \underbrace{1}_{\text{standard deviation}} \cdot \underbrace{\varepsilon}_{\text{random offset}}$$

Same offset	Before: centre 0	After: centre 1
$\varepsilon = 0.5$	$z = 0 + 0.5 = 0.5$	$z = 1 + 0.5 = 1.5$

For a general Gaussian encoder, $z = \mu + s \cdot \varepsilon$, with $\varepsilon \sim N(0, 1)$ independent of the encoder parameters.

SO **The chain rule follows parameters through the candidate to its score.**

NEXT What exactly are we trying to improve with this sampling rule?

Improve the average score of generated candidates

Yes: change the sampling rule's parameters so its candidates receive a higher average training score.

$$\text{score}(z) = \underbrace{\log p(x, z)}_{\text{model term}} - \underbrace{\log q_\varphi(z \mid x)}_{\text{encoder term}}$$

Step	What we do
Generate	Draw candidates using the current encoder
Score	Use both terms above, not only the model term
Improve	Use averaged sample gradients to adjust the encoder

Improve the **distribution's average**, not every random draw. We want a posterior distribution, not just one high-scoring hidden value.

SO **At a fixed model, a higher expected score means a smaller divergence to the posterior.**

NEXT Which direction should we change its parameters?

The gradient tells us how to improve the encoder

Keep $x = 1.6$. Our encoder guesses mean $a \cdot x$. We must learn the parameter a.

Current guess	Exact answer, for checking
Start at $a = 0$: mean $0 \cdot 1.6 = 0$	Posterior mean 0.8

Should we increase or decrease a to improve the training score?

Gradient sign	Small update that raises $\log p(x, z) - \log q_\varphi(z \mid x)$ locally
Positive	Increase a
Negative	Decrease a

SO **The gradient supplies a training direction; the exact posterior is only a check, not a training label.**

NEXT Where does $\log p(x, z)$ part of $\log p(x, z) - \log q_\varphi(z \mid x)$ come from?

The model gives two bell-shaped probabilities

Recall our assumptions: the hidden value is near zero; the observation is near the hidden value. Both variances are one.

Model assumption	Normal density
$z \sim N(0, 1)$	$p(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$
$x \mid z \sim N(z, 1)$	$p(x \mid z) = \frac{1}{\sqrt{2\pi}} e^{-(x-z)^2/2}$

Multiply: $p(x, z) = p(z) \cdot p(x \mid z) = \frac{1}{2\pi} e^{-z^2/2 - (x-z)^2/2}$.

Take logs: $\log(e^u) = u$, and products become sums. Therefore

$$\log p(x, z) = \underbrace{-\log(2\pi)}_{C} - z^2/2 - (x - z)^2/2.$$

SO **The two squares come from the two normal densities; C is constant in z.**

NEXT How do we differentiate the two terms of that log joint?

First differentiate the log-joint contribution

We need the full term $\log p(x, z) - \log q_a(z \mid x)$. First compute the derivative of its log-joint part. Keep $x = 1.6$.

> $\log p(x, z) = C - \frac{1}{2}z^2 - \frac{1}{2}(x - z)^2$, where C does not depend on z.

Term	Derivative with respect to z
Prior: $-z^2/2$	$-z$
Observation: $-(x - z)^2/2$	$x - z$
Add the derivatives	$-z + (x - z) = x - 2z$

SO **At $x = 1.6, z = 0, \log p(x, z)$ slope is $1.6 - 2 \cdot 0 = \mathbf{1.6}$.**

NEXT How does changing the parameter change the candidate and its score?

Differentiate the full sample training term

The quantity we differentiate is $S(a) = \log p(x, z(a)) - \log q_a(z(a) \mid x)$, with $z(a) = ax + s\varepsilon$. Hold x, s, ε fixed.

$$\frac{dS}{da} = \underbrace{\frac{\partial \log p(x,z)}{\partial z}\frac{dz}{da}}_{\text{log-joint contribution}} - \underbrace{\frac{d}{da}\log q_a(z(a) \mid x)}_{\text{zero for this mean-only update}} .$$

Why zero? Substitute $z(a) = ax + s\varepsilon$:

$\log q_a(z(a) \mid x) = -\frac{1}{2}\log(2\pi) - \log s - \frac{(z(a)-ax)^2}{2s^2} = -\frac{1}{2}\log(2\pi) - \log s - \frac{1}{2}\varepsilon^2$.

No a remains when s, ε stay fixed. The candidate moves with the mean; it is **not a fixed-z derivative**.

SO **The joint-only derivative equals the full derivative here because the encoder term is constant along this particular path.**

NEXT Check the two nonzero slopes, then verify why the encoder derivative is zero.

A positive gradient tells us to increase the parameter

For this arithmetic example, choose our same observation $x = 1.6$, starting parameter $a = 0$, standard deviation $s = 1$, and illustrative noise $\varepsilon = 0$. These are inputs, not derived answers.

Where the number comes from	Calculation
Sample rule: $z = a \cdot x + s \cdot \varepsilon$	$z = a \cdot x + s \cdot \varepsilon = 0 \cdot 1.6 + 1 \cdot 0 = 0$
Keep $x = 1.6, s = 1, \varepsilon = 0$ fixed; vary a	$z = 1.6a$, so $\partial z / \partial a = \mathbf{1.6}$
Previous page: score slope at $z = 0$	$\partial_z \log p(x, z) = \mathbf{1.6}$
Chain rule: multiply the two slopes	$\partial_a \log p(x, z) = 1.6 \cdot 1.6 = \mathbf{2.56}$

Why no encoder derivative here? Along $z(a) = ax + s\varepsilon$, we have $z(a) - ax = s\varepsilon$. Thus $-\log q_a(z(a) \mid x) = \frac{1}{2}\log(2\pi) + \log s + \frac{1}{2}\varepsilon^2$: **constant in** a when s, ε stay fixed. This does not hold for a spread update.

SO **This sample gives a positive slope: take a small step increasing a. Average sampled slopes for training; one sample is not a guarantee.**

NEXT How do we turn a gradient into one actual parameter update?

CLASSROOM CHECKPOINT · PREDICT, EXPLAIN, THEN REVEAL

Your turn: use the two slopes to take a step

Keep our observation 1.6. Work in pairs for three minutes; identify which numbers are chosen inputs.

Given	Your calculation
Start: a = 0, s = 1	New a = 0 + 0.12 × 2.56 = ?
Slopes: +2.56 and −1	New s = 1 + 0.12 × (−1) = ?
Chosen step size: 0.12	Does the mean move toward 0.8? Why?

Check: a becomes 0.3072; s becomes 0.88. The new mean is 0.3072 × 1.6, about 0.49, closer to 0.8.

SO **A gradient becomes learning only when we actually update the parameters.**

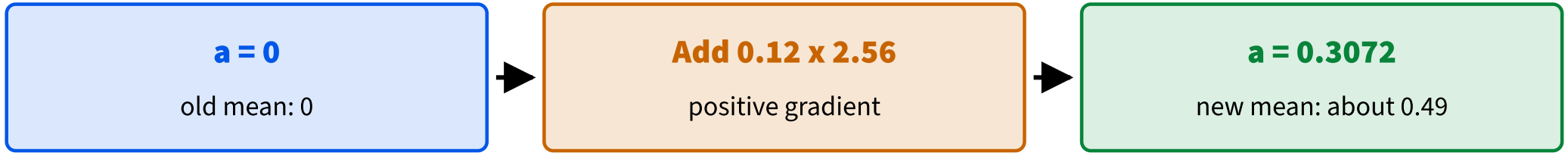
NEXT Compare your update with the fully labelled calculation.

Take one update in the direction of higher score

Now use the exact expected slopes for this checkable toy model. In a difficult model, averaged sampled slopes approximate them.

Same observation $x = 1.6$: slopes $(1 - 2a) \cdot 1.6^2$ and $1/s - 2s$. At $a = 0$, $s = 1$, these are 2.56 and -1.

Parameter	Start	Slope	Add 0.12 × slope	New value
Mean slope a	0	+2.56	+0.3072	0.3072
Standard deviation s	1	−1	−0.12	0.88



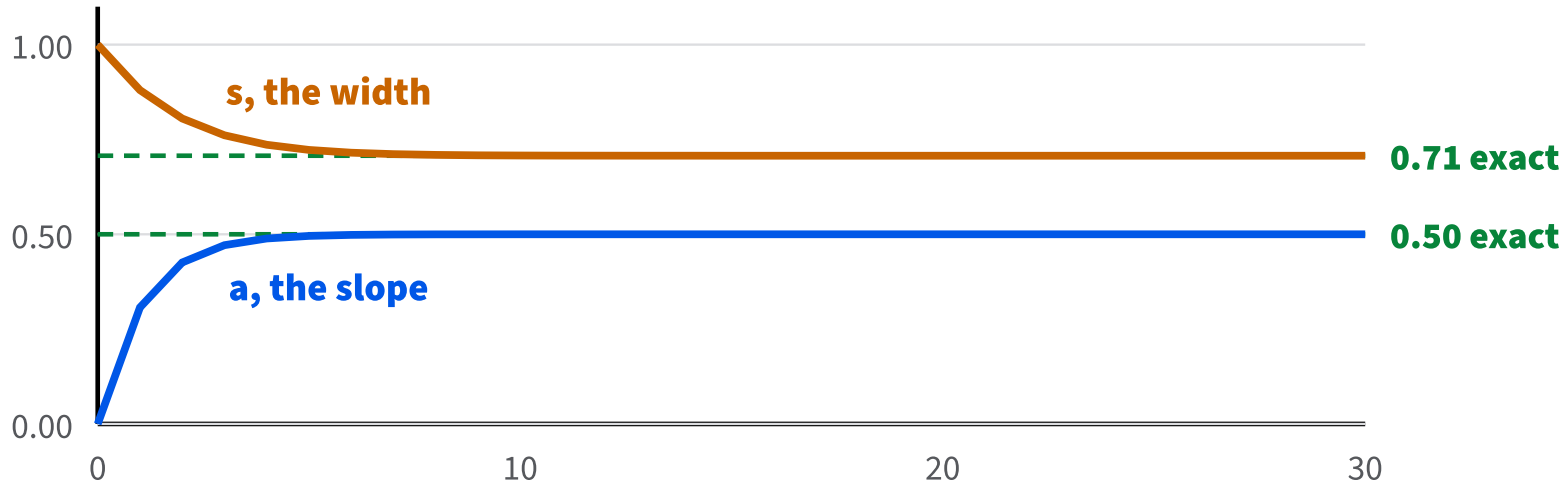
The updated mean moves toward the checkable posterior mean 0.8.

SO **Add the step size times each slope to its parameter.**

NEXT What happens when we repeat the update?

Repeated updates approach the checkable optimum

Repeat the update, recomputing the slopes after each change.



started at a = 0 and s = 1, that is at the prior; both approach the exact values

SO **The displayed run approaches the checkable analytic solution.**

NEXT Does the learned distribution match the known posterior?

After fitting, check the rule on new observations

First check our observation 1.6; then test two new observations under the same model. The values below are inputs, not new models.

Observation	Encoder mean = x/2	Posterior mean	Both variances
−1.2	−0.6	−0.6	0.5
0.4	0.2	0.2	0.5
1.6	0.8	0.8	0.5

Original input 1.6

mean 0.8

Same learned rule

mean = x / 2

New test inputs

−1.2 gives −0.6; 0.4 gives 0.2

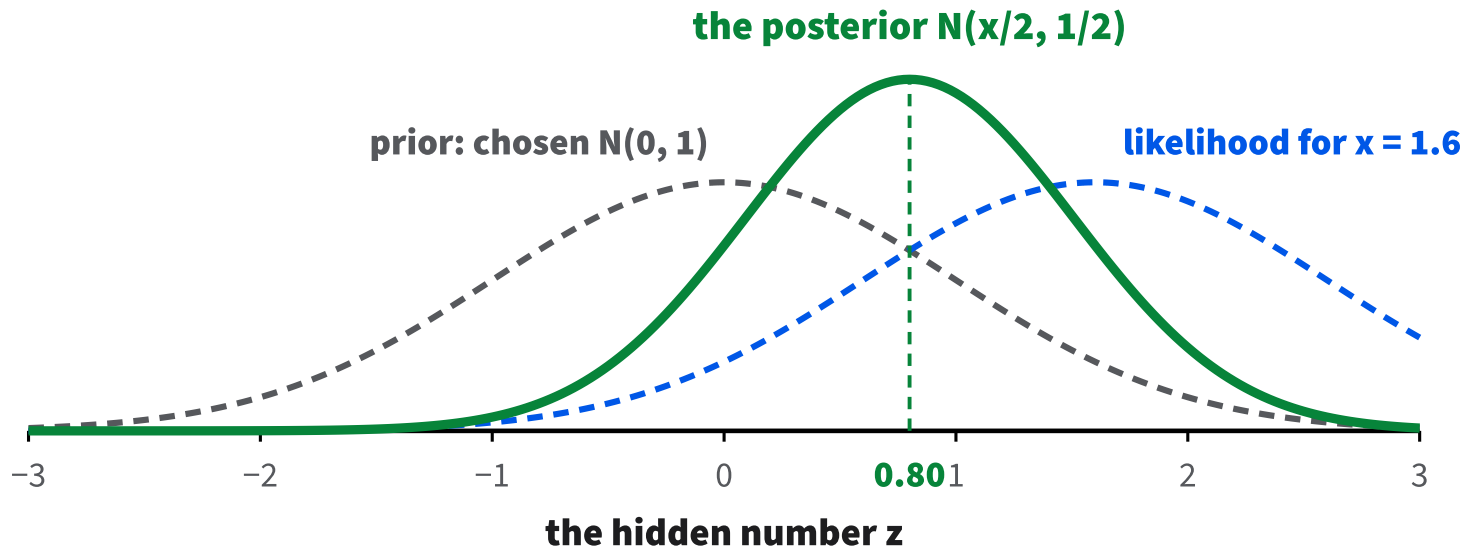
New observations test the same model and rule; they do not define a different model.

SO **A matching centre alone is insufficient; check the spread too.**

NEXT Why can the inference gap reach zero in this model?

Zero gap requires a representable and attained answer

Ask why this example can attain a **zero** inference gap.



prior variance = noise variance = 1: a model assumption

SO **Both the separate family and shared rule can express the exact answers.**

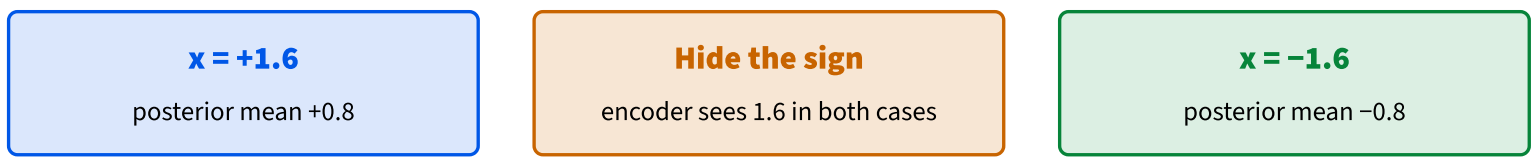
NEXT What if the shared encoder is not told the sign of the observation?

A shared encoder needs enough information

Keep the Gaussian model. Compare our observation 1.6 with a new observation -1.6. Hide their signs from the encoder.

Observation	Exact posterior mean	Encoder sees only magnitude
1.6	0.8	1.6
-1.6	-0.8	1.6

Separate Gaussian guesses can match both posteriors. A shared function receiving identical inputs must return the same guess.



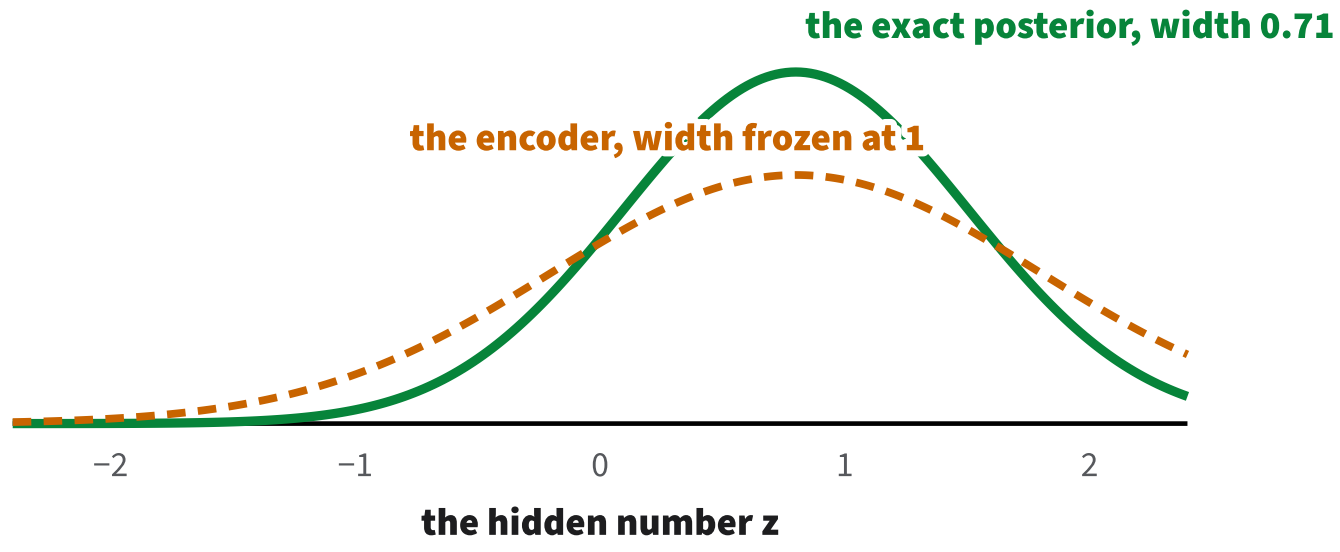
Different required answers can become indistinguishable to the restricted encoder.

SO **The missing sign creates an amortization gap; the separate distribution family is adequate.**

NEXT What if the distribution family itself forbids the correct spread?

A correct centre does not guarantee a correct guess

Restrict the spread while leaving the encoder's mean rule free.



the mean is still exact; only the width is wrong, and the bound falls by 0.153 nats

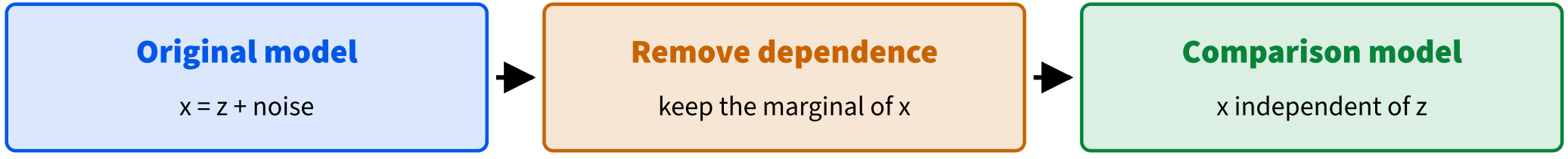
SO **An incorrect fixed variance creates an approximation gap.**

NEXT What changes if the observation no longer depends on the hidden value?

If observations stop depending on z, inference is empty

Keep the hidden prior. For this comparison, change only whether the observation depends on the hidden value.

Model	Effect of observing x
Original: $x = z + \eta$	The observation informs the hidden value
Comparison: $x \sim N(0, 2)$, independent of z	$p(z \mid x) = p(z)$: no hidden information



Only the dependence changes: observed x now says nothing about z.

SO **The comparison preserves the marginal distribution of x, but the latent now explains nothing.**

NEXT How do all these calculations connect back to the theory?

The same example has now used every theoretical tool

We kept the Gaussian model and observed value fixed until the explicitly labelled collapse comparison.

Theory	What we computed
Posterior target	Mean 0.8, variance 0.5
Encoder and gradient	Change the mean rule and spread to raise the lower bound $\mathcal{L}$
Inference gaps	Exact family versus a frozen incorrect spread
Collapse	Remove dependence on the hidden value



All three belong to the same inference task.

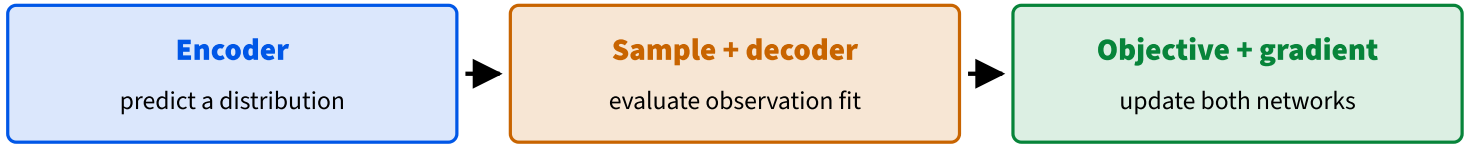
SO **We can now recognise the same roles inside larger systems.**

NEXT How does an image model use an encoder and a latent representation?

One complete training loop ties the pieces together

The numerical model let us check the answers. A neural model follows the same computational sequence.

Step	Action
1 · Encode	Read a batch; output means and positive standard deviations
2 · Sample	Draw independent noise; form $z = \mu + s \cdot \varepsilon$
3 · Score	Compute fit minus prior penalty, or the equivalent full score
4 · Differentiate	Average gradients of the batch objective
5 · Update	Adjust encoder parameters and, when learned, decoder parameters



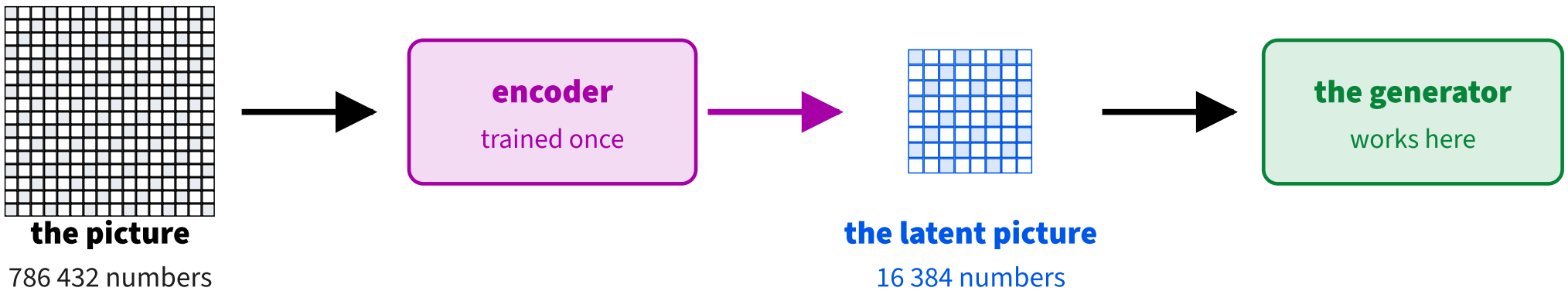
During generation, start from the prior instead of an existing observation.

SO **The toy decoder was fixed; a full variational autoencoder learns both networks.**

NEXT Where do these components appear in practical systems?

The hidden representation can make generation cheaper

Transfer the idea from **hidden variable** choices to hidden image representations.



the slow part of the system never sees a pixel: it sees the encoder's output

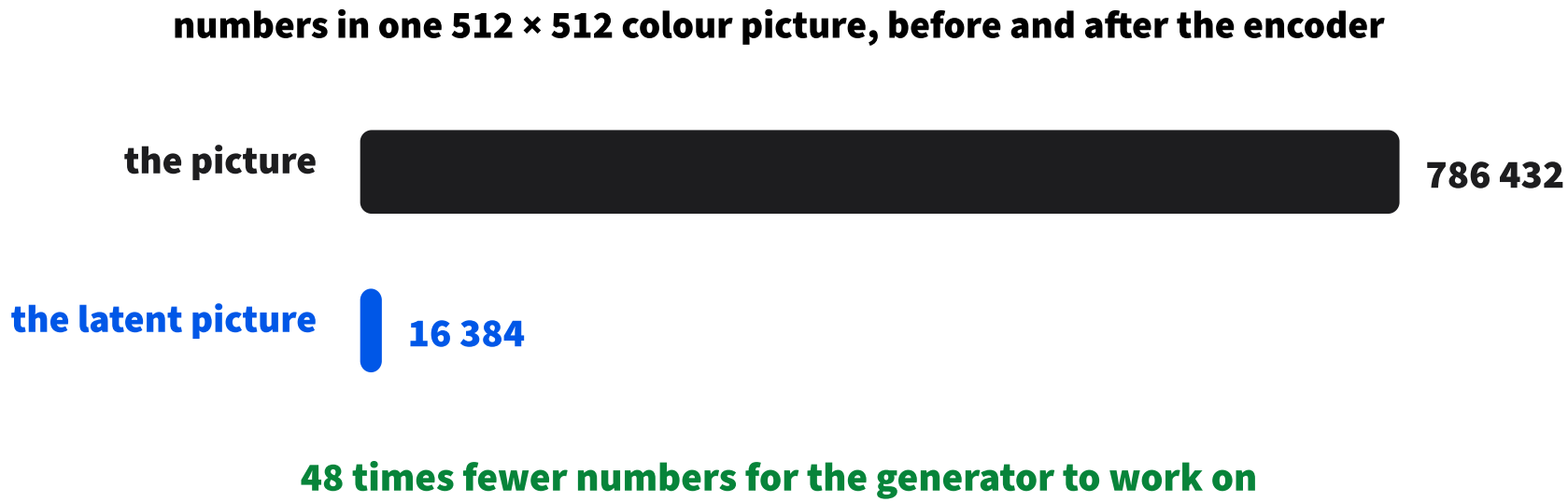
and a decoder turns the result back into a picture

SO **An encoder and decoder connect pixels to a smaller latent space.**

NEXT Count the entries in one specific image and latent representation.

Count the image entries before comparing costs

Count the entries in one specific image and **latent representation**.

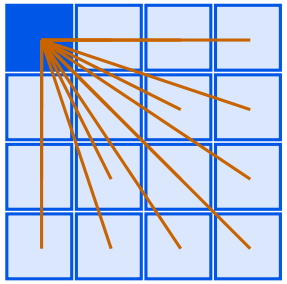


SO **This design reduces the number of entries by a factor of 48.**

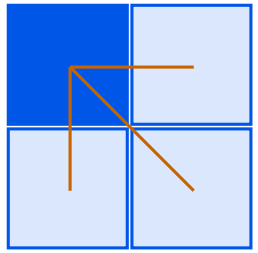
NEXT A smaller grid changes the number of positions an attention layer compares.

Attention compares each position with every position

A **position** is one grid cell. In full attention, each cell compares itself with **all cells**, including itself.



one blue query looks at all 16 positions



one blue query looks at all 4 positions

Repeat for every query position: N queries, each comparing N positions.

Small illustration	Positions N	Full-attention comparisons N × N
4 × 4 grid	16	16 × 16 = 256
2 × 2 grid	4	4 × 4 = 16

SO **Four times fewer positions gives 16 times fewer pair comparisons. This counts attention pairs, not total generation speed.**

NEXT What information must the encoder retain so the decoder can rebuild an image?

Keep image details while regularising the hidden code

An image encoder produces a smaller hidden code. Its decoder must use that code to rebuild the original image's details.

Training requirement	In plain words
Reconstruction quality	Keep enough image information for the decoder
Prior penalty $\mathrm{KL}(q(z \mid x) \| p(z))$	Keep each guessed code distribution near the prior
Weight the two requirements	Decide how strongly to regularise while preserving useful details

> A simplified weighted objective: $\text{reconstruction reward} - \beta\,\mathrm{KL}(q(z \mid x) \| p(z))$. Here β sets the penalty weight.

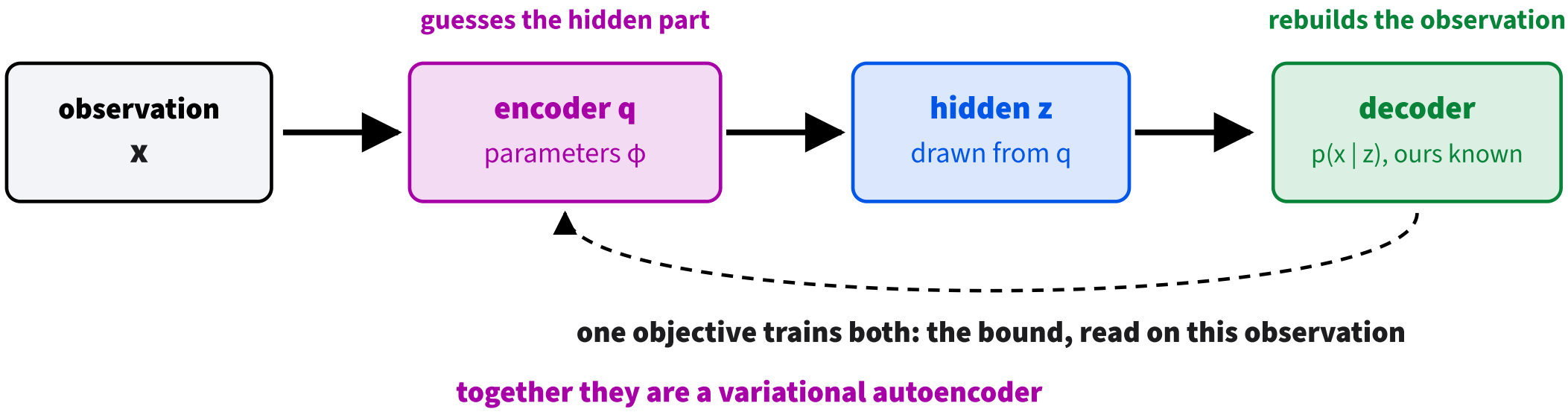
If every image produced the same code distribution, the code would not distinguish their details. A weaker penalty may help retain information; it is not a guarantee. Image systems can also use perceptual and adversarial losses.

SO **Image autoencoders balance useful reconstruction and latent regularisation. A modified loss need not be the original lower bound $\mathcal{L}$.**

NEXT Where else do we reuse a learned function to infer hidden information?

The reusable idea is a learned inference function

The same hidden-value inference task also appears when experiments can be simulated.



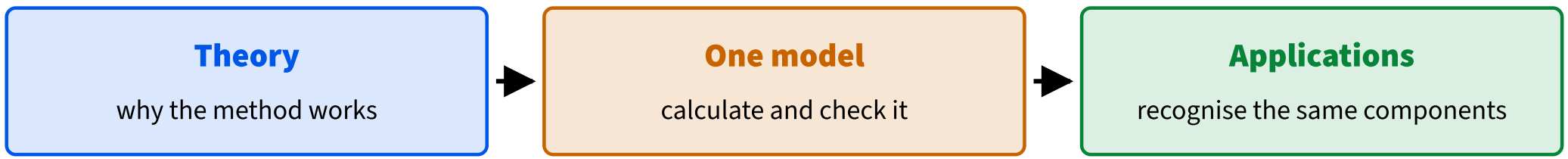
SO **A learned posterior proposal still needs calibration and distribution checks.**

NEXT Review each obstacle and the tool introduced to answer it.

The whole method starts with a hidden cause

Follow the path from repeated Bayes calculations to one trained inference function.

We start with	We do	We check
Observed data	Infer a hidden cause	Does the guess explain the observation?
Many similar cases	Train one encoder	Are its inputs sufficient?
A fitted encoder	Reuse on new cases	Are its guesses accurate and informative?



Return to the same question: what hidden information does the encoder infer?

SO **Useful inference needs adequate inputs, expressive guesses and a trainable objective.**

NEXT Test the distinctions that determine when the shortcut helps or fails.

CHECK YOURSELF

What prevents the encoder from matching Bayes?

Test the distinctions that determine when the shortcut helps or fails.

1. An encoder reads h and n and still has a gap. What has gone wrong?

the inputs | the family or the training | nothing: that is impossible

Answer. The family or the training: with both inputs the **posterior** map is reachable (p. 118), so a gap now means the family is too small or the fit is not at its best.

SO **Explain a restriction's cause before choosing a remedy.**

NEXT Why can training remain noisy even with a correct average **gradient**?

CHECK YOURSELF

Why can a correct gradient still be noisy?

A sampled **gradient** can fluctuate even when its average is correct.

2. Why can $\log p(x, z) - \log q_\varphi(z \mid x)$-function gradient become noisy?

it uses more memory | one score weights every coordinate | it is biased

Answer. It judges every coordinate by one score for the whole draw, so the **noise** of that score enters every coordinate (p. 124); it stays **correct on average**.

SO **An unbiased estimate can still have high variance.**

NEXT Does a **zero** prior penalty mean the posterior guess is exact?

CHECK YOURSELF

Does a zero prior penalty mean a tight bound?

Prior penalty $\mathrm{KL}(q\|p(z))$ is different from posterior error $\mathrm{KL}(q\|p(z \mid x))$.

If $\mathrm{KL}(q(z \mid x)\|p(z)) = 0$ for every input, what does that tell us?

good: the lower bound $\mathcal{L}$ is tight | possibly uninformative: check the model | it depends on the learning rate

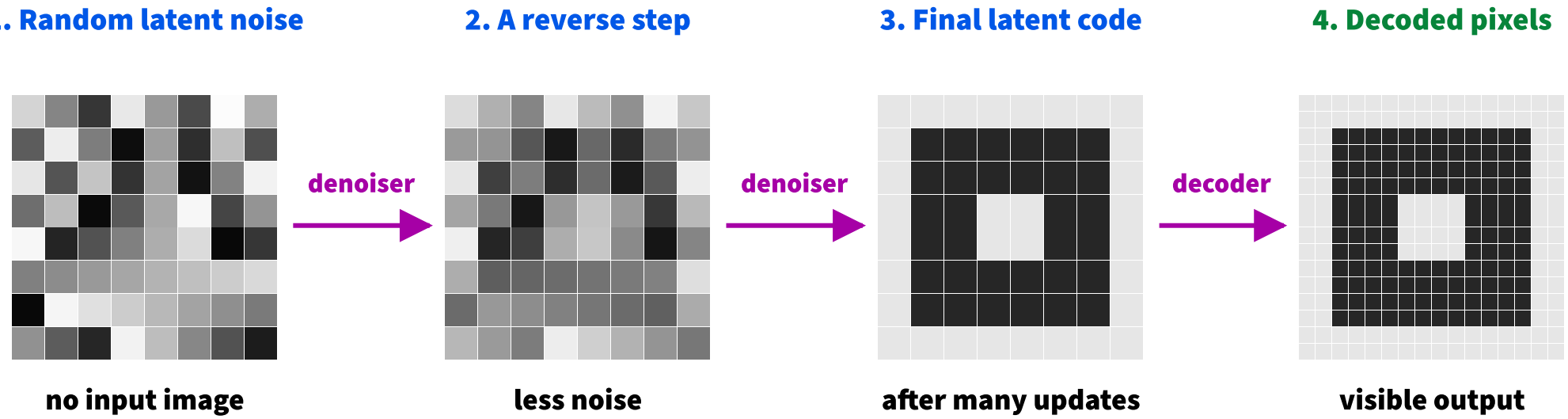
Choose "possibly uninformative: check the model". Zero prior penalty means $q(z \mid x) = p(z)$. Original model $x = z + \eta$, with $x = 1.6$: the posterior is $N(0.8, 0.5)$, so returning prior $N(0, 1)$ is incorrect. In the explicitly independent comparison model, $p(z \mid x) = p(z) = N(0, 1)$, so returning that prior is exact for that model.

SO **Zero prior penalty means $q = p(z)$; it does not by itself mean $q = p(z \mid x)$.**

NEXT The continuous example inferred a clean hidden value behind noise.

Diffusion removes noise over several learned steps

Generation starts from random latent noise. A trained denoiser repeatedly updates it; the decoder then maps the result to pixels.



The same trained denoiser is reused across noise levels; decoding is a separate operation.

Schematic grids illustrate the stages; these are not outputs of a trained diffusion model.

SO **The denoiser generates a latent representation through repeated updates; the decoder turns the final representation into an image.**

NEXT Next lecture: how is this denoiser trained, and what does each reverse step predict?

Selected references (1 of 2): the method

Kingma, D. P. and Welling, M. (2014).
Auto-encoding variational Bayes.
International Conference on Learning Representations (ICLR). arXiv:1312.6114.

Rezende, D. J., Mohamed, S., and Wierstra, D. (2014).
Stochastic backpropagation and approximate inference in deep generative models.
International Conference on Machine Learning (ICML), PMLR 32:1278–1286.

Gershman, S. and Goodman, N. (2014).
Amortized inference in probabilistic reasoning.
Proceedings of the Annual Meeting of the Cognitive Science Society, 36.

Ranganath, R., Gerrish, S., and Blei, D. M. (2014).
Black box variational inference.
Artificial Intelligence and Statistics (AISTATS), PMLR 33:814–822.

Cremer, C., Li, X., and Duvenaud, D. (2018).
Inference suboptimality in variational autoencoders.
International Conference on Machine Learning (ICML), PMLR 80:1078–1086.

Mohamed, S., Rosca, M., Figurnov, M., and Mnih, A. (2020).
Monte Carlo gradient estimation in machine learning.
Journal of Machine Learning Research, 21(132):1–62.

Selected references (2 of 2): limits and uses

Bowman, S. R., Vilnis, L., Vinyals, O., Dai, A. M., Jozefowicz, R., and Bengio, S. (2016).
Generating sentences from a continuous space.
Conference on Computational Natural Language Learning (CoNLL), 10–21.

Kingma, D. P., Salimans, T., Jozefowicz, R., Chen, X., Sutskever, I., and Welling, M. (2016).
Improved variational inference with inverse autoregressive flow.
Advances in Neural Information Processing Systems 29.

Marino, J., Yue, Y., and Mandt, S. (2018).
Iterative amortized inference.
International Conference on Machine Learning (ICML), PMLR 80:3403–3412.

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. (2022).
High-resolution image synthesis with latent diffusion models.
IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 10684–10695.

Xu, M., Quiroz, M., Kohn, R., and Sisson, S. A. (2019).
Variance reduction properties of the reparameterization trick.
Artificial Intelligence and Statistics (AISTATS), PMLR 89:2711–2720.

Johnson, M. J., Duvenaud, D., Wiltschko, A. B., Datta, S. R., and Adams, R. P. (2016).
Composing graphical models with neural networks for structured representations and fast inference.
Advances in Neural Information Processing Systems 29.

Cranmer, K., Brehmer, J., and Louppe, G. (2020).
The frontier of simulation-based inference.
Proceedings of the National Academy of Sciences, 117(48):30055–30062.

REFERENCES

Course materials informing this lecture

These are teaching references, not claims that all courses use one identical sequence.

Course	Relevant material
Stanford CS231n	Generative Modeling: probabilistic encoders, objectives, generation
Stanford CS236	VAE notes: latent models, lower bounds, gradients, amortization
CMU 10-708, Lecture 17	Connections between generative and inference models
Berkeley CS294-158, Spring 2024	Latent-variable-model lecture and implementation homework

SO **Our teaching choice remains theory → one worked example → applications.**

NEXT Use the appendix for optional derivations and earlier coin comparisons.

Thank you

Laboratory: Friday 13 November · Next lecture: Thursday 19 November

Appendix: the three derivations, the coin logit, the gap table and the Gaussian bound

The evidence splits into the evidence lower bound and a divergence

↳ Recalled on **(p. 107)**. Here q is any distribution over z with $q(z) > 0$ wherever $p(z \mid x) > 0$.

$$\log p(x) = \log \int p(x, z)\, dz$$

the evidence, by definition

$$= \log \int q(z)\, \frac{p(x, z)}{q(z)}\, dz$$

multiply and divide by $q(z)$

$$= \mathbb{E}_q\left[\log \frac{p(x, z)}{q(z)}\right] + \mathbb{E}_q\left[\log \frac{q(z)}{p(z \mid x)}\right]$$

write $p(x, z) = p(z \mid x)\, p(x)$

$$= \mathcal{L}(q) + \mathrm{KL}\big(q \,\|\, p(z \mid x)\big)$$

name the two averages

In words: why that ratio: $p(x, z)/q(z) = p(x) \cdot p(z \mid x)/q(z)$. If q is the posterior the ratio is $p(x)$ for every z, so its average is exactly $\log p(x)$; any other q makes the ratio vary, and the average of a varying log falls short.

The divergence is never negative, so $\mathcal{L}(q)$ is a floor, equal only at the posterior.

Sharing one function adds a second gap

↳ The two gaps of (p. 117), written out. $\mathcal{Q}$ is the family of distributions the **encoder** can output.

$$\log p(x) - \mathcal{L}(q_\varphi(\cdot \mid x)) = \mathrm{KL}\big(q_\varphi(\cdot \mid x) \,\|\, p(z \mid x)\big)$$

one observation, from (p. 66)

$$\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q \,\|\, p(z \mid x)\big) \;\le\; \mathrm{KL}\big(q_\varphi(\cdot \mid x) \,\|\, p(z \mid x)\big)$$

the **encoder** is one member of $\mathcal{Q}$

$$\text{approximation}(x) = \min_{q \in \mathcal{Q}} \mathrm{KL}, \qquad \text{amortization}(x) = \text{the difference}$$

name the two parts

$$\mathbb{E}_x\big[\text{approximation}\big] + \mathbb{E}_x\big[\text{amortization}\big] = \log p(x)\text{'s shortfall}$$

average over the data

Richer family: first part falls. More inputs or a better fit: second part falls.

Why the gradient may be taken inside the average

↳ Used on **(p. 35)**, for $q_\varphi = N(\mu_\varphi, s_\varphi^2)$ and any smooth f.

$$z = \mu_\varphi + s_\varphi \cdot \varepsilon, \qquad \varepsilon \sim N(0,1)$$

the same law as $z \sim q_\varphi$

$$\mathbb{E}_{z\sim q_\varphi}[f(z)] = \mathbb{E}_{\varepsilon\sim N(0,1)}\left[f(\mu_\varphi + s_\varphi\varepsilon)\right]$$

change of variable

$$\nabla_\varphi \mathbb{E}_\varepsilon[\,\cdot\,] = \mathbb{E}_\varepsilon\left[\nabla_\varphi f(\mu_\varphi + s_\varphi\varepsilon)\right]$$

the law of ε is free of φ

$$= \mathbb{E}_\varepsilon\left[f'(z)\cdot(\nabla_\varphi\mu_\varphi + \varepsilon\cdot\nabla_\varphi s_\varphi)\right]$$

chain rule through the sample

One draw of ε gives an unbiased gradient, and it uses f', not just f.

The estimator that needs no reparameterization

↳ The alternative of **(p. 123)**; it works for any q_φ, including discrete ones.

$$\nabla_\varphi \int q_\varphi(z) f(z)\, dz = \int \nabla_\varphi q_\varphi(z) \cdot f(z)\, dz$$

differentiate the density

$$\nabla_\varphi q_\varphi(z) = q_\varphi(z) \cdot \nabla_\varphi \log q_\varphi(z)$$

because $\nabla \log u = \nabla u / u$

$$= \mathbb{E}_{z\sim q_\varphi}\left[f(z) \cdot \nabla_\varphi \log q_\varphi(z)\right]$$

an average again, so sample it

Correct on average, but one number $f(z)$ is charged to every coordinate of φ.

Why the coin posterior is exactly $\sigma(a + b \cdot h)$

↳ The claim of **(p. 101)**, for n tosses of a coin that is A with probability π.

$$\frac{P(A \mid h)}{P(B \mid h)} = \frac{\pi \cdot \theta_A^h (1-\theta_A)^{n-h}}{(1-\pi) \cdot \theta_B^h (1-\theta_B)^{n-h}}$$

Bayes' rule; the counts cancel

$$\log \frac{P(A \mid h)}{P(B \mid h)} = \log \frac{\pi}{1-\pi} + h \cdot \log \frac{\theta_A}{\theta_B} + (n-h) \cdot \log \frac{1-\theta_A}{1-\theta_B}$$

take logarithms

$$= \underbrace{\log \frac{\pi}{1-\pi} + n \cdot \log \frac{1-\theta_A}{1-\theta_B}}_{a} + h \cdot \underbrace{\log \frac{\theta_A(1-\theta_B)}{\theta_B(1-\theta_A)}}_{b}$$

collect the terms in h

$$n = 2: \quad a = \log \tfrac{1}{9} = -2.197, \qquad b = \log 6 = 1.792$$

our two coins, 0.8 and 0.4

a carries n; an encoder blind to n is therefore blind to part of a.

The 0.078 nats, case by case

tosses	h	how often	exact	encoder	divergence, nats
2	0	0.100	0.100	0.089	0.001
2	1	0.200	0.400	0.264	0.041
2	2	0.200	0.800	0.569	0.138
4	0	0.033	0.012	0.089	0.103
4	1	0.093	0.069	0.264	0.181
4	2	0.125	0.308	0.569	0.145
4	3	0.141	0.727	0.829	0.029
4	4	0.109	0.941	0.947	0.000
average, each row weighted by how often it happens					**0.078**

Two tosses and four tosses are pulled to one answer, and the middle counts pay for it.

The evidence lower bound for the Gaussian bag, and its two slopes

↳ Used on (p. 43). Model: $z \sim N(0,1)$, $x = z + \text{noise}$; **encoder** $q = N(a \cdot x,\ s^2)$.

$$\mathcal{L} = \mathbb{E}_q[\log p(z)] + \mathbb{E}_q[\log p(x \mid z)] + \mathbb{H}[q]$$

the lower bound $\mathcal{L}$, in three parts

$$= -\tfrac{1}{2}\left(a^2x^2 + s^2\right) - \tfrac{1}{2}\left((1-a)^2x^2 + s^2\right) + \tfrac{1}{2}\log s^2 + \text{const}$$

each part in closed form

$$\frac{\partial \mathcal{L}}{\partial a} = (1-2a)\cdot x^2, \qquad \frac{\partial \mathcal{L}}{\partial s} = \frac{1}{s} - 2s$$

differentiate; average x^2 over the data

zero slopes: $a = \tfrac{1}{2}, \quad s^2 = \tfrac{1}{2}$

the **exact posterior** $N(x/2, 1/2)$

At $a = 0$, $s = 1$ and $x = 1.6$: slopes $+2.56$ and -1.00, as on (p. 43).

Why equal coins force a flat encoder

↳ The claim of (p. 127), with both coins at $\theta_A = \theta_B = \theta$.

$$\log \frac{P(A \mid h)}{P(B \mid h)} = \log \frac{\pi}{1-\pi} + h \cdot \log \frac{\theta}{\theta} + (n-h) \cdot \log \frac{1-\theta}{1-\theta}$$

from (p. 70)

$$= \log \frac{\pi}{1-\pi} = 0 \quad \text{for every } h$$

both logarithms are **zero**; $\pi = \frac{1}{2}$

$$\arg\min_{a,b} \; \mathbb{E}_h \, \mathrm{KL}\big(\sigma(a+bh) \,\|\, \tfrac{1}{2}\big) \;\Rightarrow\; a = b = 0$$

the divergence is **zero** only at $\frac{1}{2}$

In words: **In words: the two coins differ in nothing, so no count of heads can favour one of them, and the best answer is the prior itself.**

↳ In a variational autoencoder the same flat answer has another cause: the **decoder** stops needing z (p. 129).

The encoder is right to say nothing: here the observation carries nothing to say.

Why conjugacy makes the exact encoder affine

↳ The theorem of (p. 103), with our two models as its instances.

$p(z) \propto \exp\big(\langle \lambda,\, t(z)\rangle\big), \qquad p(x \mid z) \propto \exp\big(\langle u(x),\, t(z)\rangle\big)$

a conjugate pair: the same $t(z)$

$p(z \mid x) \propto p(z)\, p(x \mid z) \propto \exp\big(\langle \lambda + u(x),\, t(z)\rangle\big)$

multiply; the exponents add

coins: $t(z) = 1[z = A], \quad u(x) = h \log \frac{\theta_A}{\theta_B} + (n-h) \log \frac{1-\theta_A}{1-\theta_B}$

affine in (h, n) (p. 70)

bells: $t(z) = \big(z, -\frac{1}{2}z^2\big), \quad u(x) = (x,\, 1) \;\Rightarrow\; \text{posterior mean } x/2$

affine in x (p. 25)

In words: **In words: the observation adds to the prior's parameter instead of reshaping it, and that sum is what an affine encoder gives.**

↳ Johnson, Duvenaud, Wiltschko, Datta and Adams (2016) build encoders on **exactly** this.

Both exact encoders of today are one theorem, written twice.

The approximation result behind “a larger encoder”

↳ The theorem of **(p. 119)**, with the conditions it needs and where it comes from.

condition	why it is needed
the observations lie in a bounded, closed set	the approximation is uniform only on such a set
the map from x to the best parameters of q is continuous	a network is continuous, so its target must be
one hidden layer, any width, a non-polynomial activation	the form in which the theorem is stated
what it does not give	the width, the training, or the approximation gap

Sources: Cybenko (1989), *Mathematics of Control, Signals and Systems* 2:303–314; Hornik (1991), *Neural Networks* 4(2):251–257. That most of a measured gap is amortization is the finding of Cremer, Li and Duvenaud (2018).

Existence on a bounded set: enough to justify a bigger encoder, not to size it.

Why neither estimator pulls training the wrong way

↳ The unbiasedness of **(p. 125)**; both lines exchange a derivative with an integral.

$$\mathbb{E}_{\varepsilon}\left[\nabla_{\varphi} f(\mu_{\varphi} + s_{\varphi}\varepsilon)\right] = \nabla_{\varphi}\,\mathbb{E}_{\varepsilon}\left[f(\mu_{\varphi} + s_{\varphi}\varepsilon)\right]$$

through the sample **(p. 68)**

$$\mathbb{E}_{q_{\varphi}}\left[f(z)\,\nabla_{\varphi}\log q_{\varphi}(z)\right] = \nabla_{\varphi}\,\mathbb{E}_{q_{\varphi}}[f(z)]$$

the guessed **gradient (p. 69)**

$$\varphi_{t+1} = \varphi_t + \alpha_t g_t, \quad \mathbb{E}[g_t] = \nabla\mathcal{L}(\varphi_t), \quad \textstyle\sum_t \alpha_t = \infty,\ \sum_t \alpha_t^2 < \infty$$

Robbins and Monro (1951)

$$\Rightarrow \quad \nabla\mathcal{L}(\varphi_t) \to 0 \ \text{ along a subsequence}$$

requires additional convergence assumptions

In words: **In words: a single step may point anywhere; the average is the gradient when differentiation and expectation may be interchanged. Convergence additionally needs smoothness, bounded noise and stable iterates.**

↳ The survey of both estimators is Mohamed, Rosca, Figurnov and Mnih (2020), *JMLR* 21(132).

Correctness needs only the average; the variance decides how long it takes.

What maximizing the evidence lower bound guarantees about the model

↳ The guarantee of (p. 110), with θ trained as well.

$$\log p_\theta(x) = \mathcal{L}(\theta, \varphi; x) + \mathrm{KL}\big(q_\varphi(\cdot \mid x) \,\|\, p_\theta(z \mid x)\big)$$

the split of (p. 66), with θ

$$\Rightarrow \quad \log p_\theta(x) \;\geq\; \mathcal{L}(\theta, \varphi; x) \;\text{ for every } \varphi$$

the divergence is never negative

$$\max_{\theta,\varphi} \mathcal{L} \;\leq\; \max_{\theta} \log p_\theta(x)$$

so the lower bound $\mathcal{L}$ never overstates

$$\log p_\theta(x) - \mathcal{L}(\theta, \varphi; x) = \mathrm{KL}(q_\varphi \| p_\theta(z \mid x))$$

the gap at the same model parameters

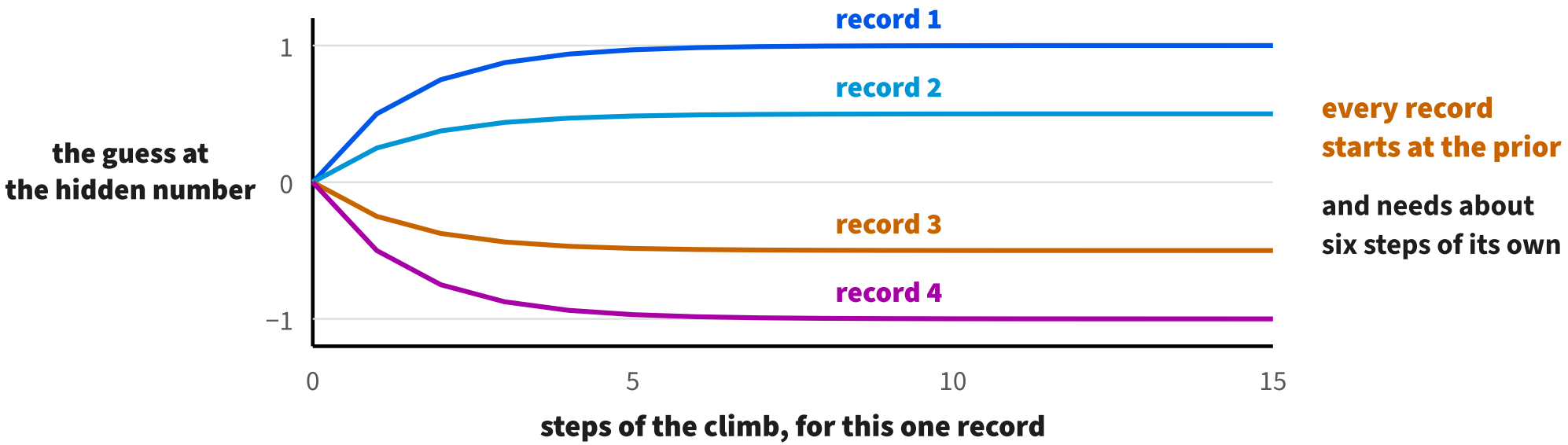
In words: In words: the reported score is always achieved, so the lower bound $\mathcal{L}$ is a lower score at each fixed model. Increasing it while changing the model need not increase log evidence.

↳ That is why a bound is safe to maximize at all.

A lower bound is not a monotonic likelihood guarantee.

Six steps per record, when the hidden part is a number

↳ The claim of **(p. 96)**, measured on the model of Part 5 **(p. 122)**: a hidden number $z \sim N(0, 1)$ and a record $x = z + \text{noise}$. Four records, each climbing the lower bound $\mathcal{L}$ from the **prior**.



Each record repeats the whole climb, and learns nothing from the records before it.

CALCULATION · (P. 115)

One head means different things in different bags

Coin A favours heads. One head and three tails therefore argues against A more strongly than one head and one tail.

For two tosses, the A and B joint probabilities are $0.16, 0.24$, so the **posterior** is $0.16/(0.16 + 0.24) = 0.40$.

For four tosses, there are four positions for the single head. Multiplying **prior**, positions and toss probabilities gives $0.5 \cdot 4 \cdot 0.8 \cdot 0.2^3 = 0.0128$ and $0.5 \cdot 4 \cdot 0.4 \cdot 0.6^3 = 0.1728$. Thus A has probability $0.0128/0.1856 \approx 0.069$.

Reading both counts removes this input conflict

Give the **encoder** both h, the heads, and n, the tosses. The two bags now have different inputs.

The **exact posterior** can be written $q(A \mid h, n) = \sigma(h \log 6 + n \log(1/3))$, where $\sigma(t) = 1/(1 + e^{-t})$. A head multiplies the A/B odds by $0.8/0.4 = 2$; a tail multiplies them by $0.2/0.6 = 1/3$.

A **shared encoder** with this form can fit every bag **exactly**. Complete inputs make this possible; successful training is still needed.

CALCULATION · (P. 119)

More capacity helps only after inputs are sufficient

A larger network still **cannot distinguish** two bags if both arrive as the **same input**.

After including heads and tosses, a sufficiently expressive network can approximate the **posterior** map on a bounded input set.

For finite coins, continuity and probabilities bounded away from **zero** allow a small probability error to imply a small Kullback–Leibler (KL) gap. This is an existence result, **not a training guarantee**.

The observation shifts our guess toward its value

The **prior** favours a hidden value near **zero**. The noisy observation favours a hidden value near the observed number.

With equal **noise** variances, the **posterior** mean lies halfway: for $x = 2$, the mean is 1.

Completing the square gives $p(z \mid x) = N(x/2, 1/2)$. Here $1/2$ is the **variance**; the **standard deviation** is $\sqrt{1/2}$, not half the original width.

Keep the noise fixed while changing the sample

Let the **encoder** choose mean μ and **standard deviation** s. Draw independent standard-normal **noise** ε.

Compute $z = \mu + s \cdot \varepsilon$. For $\mu = 1, s = 2, \varepsilon = 0.5$, the sample is $1 + 2 \cdot 0.5 = 2$.

With that **noise** held **fixed**, changing μ moves the sample smoothly. The **chain rule** can now differentiate through z. This exact construction is for continuous Gaussian samples, not coin identities.

CALCULATION · (P. 125)

Unbiased gradients need not reach the best solution

One sampled update can point downhill, just as one coin toss **need not** match its long-run frequency.

Under conditions permitting differentiation inside the expectation, the average estimated **gradient** equals the **true gradient**. See the appendix proof.

Convergence needs additional assumptions on smoothness, **noise**, stability and step sizes. Unbiasedness alone guarantees neither convergence nor a global optimum.

CALCULATION · (P. 127)

Identical coins cannot be distinguished by tosses

Choose A or B equally often. Give both coins head probability 0.6. Observe one head in two tosses.

Each likelihood is $2 \cdot 0.6 \cdot 0.4 = 0.48$. Each joint probability is $0.5 \cdot 0.48 = 0.24$.

Bayes gives $p(A \mid x) = 0.24/(0.24 + 0.24) = 0.5$. The observation leaves our original equal probabilities unchanged.

The lower bound identifies better encoder parameters

Keep the same observed value $x = 1.6$, so $x^2 = 2.56$. No average over different observations is used here.

Per observation, $\log p(x, z) - \log q_\varphi(z \mid x)$ is $\mathcal{L} = C - \frac{1}{2}\mathbb{E}_q[(x - z)^2 + z^2] + \log s$, where $C = (1 - \log(2\pi))/2$.

Using the **encoder** mean and **variance** gives $\log p(x, z) - \log q_\varphi(z \mid x)$ for this fixed observation $J(a, s) = C - 1.28[(1 - a)^2 + a^2] - s^2 + \log s$. Expand the two squares in the appendix.

A correct centre does not guarantee a correct guess

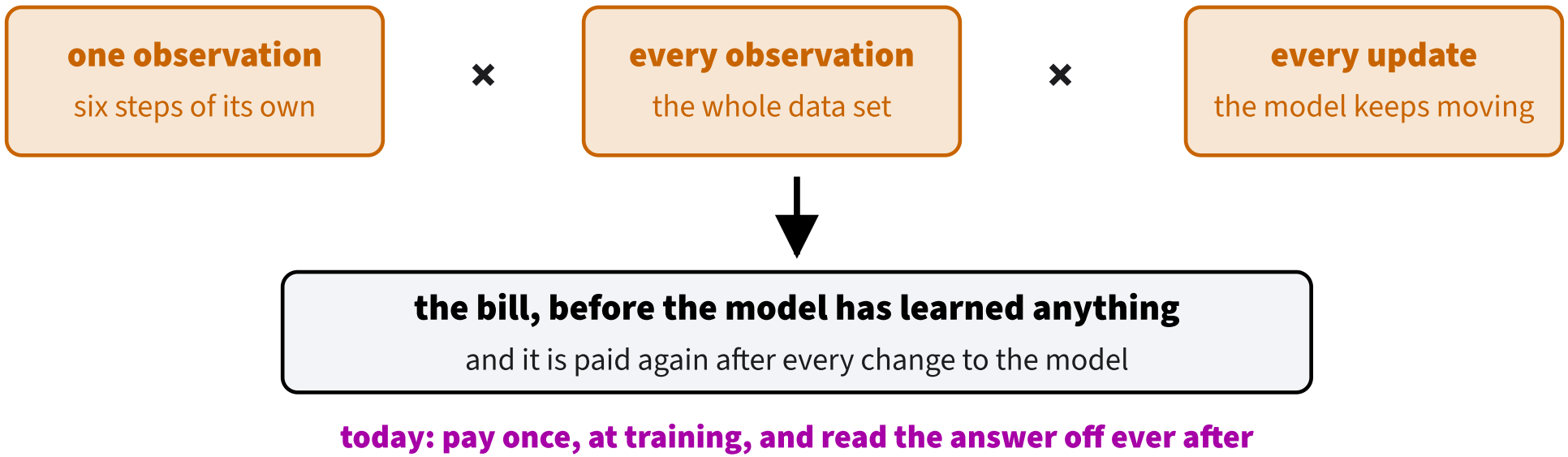
Force the **encoder standard deviation** to remain $s = 1$. It can match the **posterior** centre, but its **variance** is twice the true variance.

For matching Gaussian means, the divergence is $\frac{1}{2}[1/(1/2) - 1 + \log((1/2)/1)] = \frac{1}{2}(1 - \log 2) \approx 0.153$.

Even a separate best guess for each observation cannot fix that spread. This is an **approximation gap**, rather than an extra cost of sharing an **encoder**.

Inference is a bill we pay again after every update

We already know how to infer a hidden cause for one observation.



Today: stop paying per record. Train one function that **outputs the answer**, once and for all.

SO **Repeated observations repeat the same inference work.**

NEXT Our goal is to replace repeated inference with one reusable function.

Two results from earlier lectures that return today

1. In the E-step of expectation–maximization (Lecture 5, p. 33), what is computed for each record?

the parameters | the posterior over the hidden variable | the likelihood

Answer. The **posterior** over the **hidden variable**, recomputed for every record in every round. **Returns in Part 1**, as the work we want to stop repeating.

2. The evidence lower bound (Lecture 6, p. 9) is a bound on what, and how large is the gap?

on the posterior; the gap is the entropy | on $\log p(x)$; the gap is a divergence | on the parameters; the gap is zero

Answer. On $\log p(x)$, from below; the gap is the Kullback–Leibler divergence $\mathrm{KL}(q \,\|\, p(z \mid x))$. **Returns in Part 3**, as what we train the network to maximize.

By the end of today, you can do six things

The lecture follows one question: can inference become a learned function?

1**compute** a **posterior** by hand and use it as a targetthe target
2**write** an **encoder** that answers every observation at onceamortization
3**state** what the **evidence lower bound** is maximized overthe objective
4**measure** the **amortization gap** and name what causes itthe cost
5**explain** why we sample as $\mu + \sigma \cdot \varepsilon$the gradient
6**recognise** a hidden variable that has stopped carrying anythingcollapse

SO **Explain the shortcut, its training objective and its limitations.**

NEXT Before following the bags, distinguish observations, hidden choices and learned parameters.

Each symbol today, with its meaning in words

Before following the bags, distinguish observations, hidden choices and learned parameters.

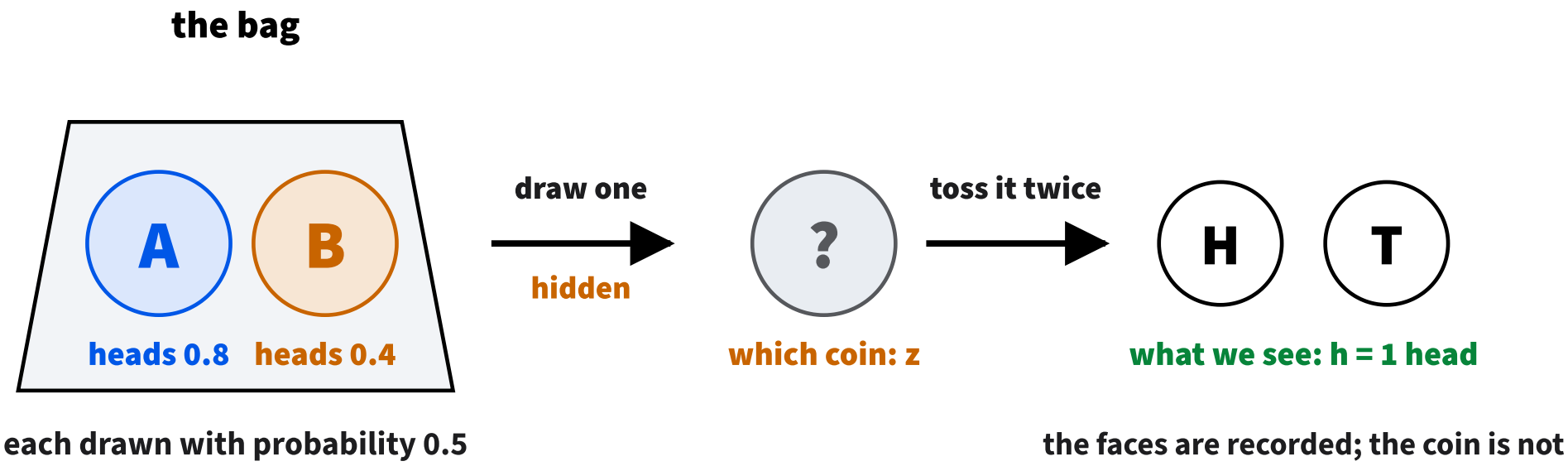
symbol	meaning
z	**the hidden variable; today, which coin was drawn: A or B**
x, h, n	**the observation; for us h heads in n tosses, so $x = (h, n)$**
$p(z), p(x \mid z)$	**the model: how the hidden part is drawn, and what it produces**
$p(z \mid x)$	**the exact posterior: what the observation says about the hidden part**
$q_\varphi(z \mid x)$	**the encoder: a function from observation to a distribution over z**
φ **(phi)**	**the encoder's parameters; ours: $\varphi = (a, b)$, two numbers**
θ **(theta)**	**Model parameters: fixed initially; learned with the decoder later.**
$\sigma(t)$	$1/(1 + e^{-t})$**: turns a score into a probability**
$\mathcal{L}(\varphi)$	**the evidence lower bound, the quantity we maximize**
$\mathrm{KL}(q \,\|\, p)$	**the Kullback–Leibler divergence: how far q is from p, in nats (Lecture 3, p. 60); zero only when $q = p$**

SO **The observation is given; the hidden variable is what we infer.**

NEXT We choose a hidden coin , toss it and record the visible result.

One bag, two coins, and a face we can see

We choose a **hidden coin**, toss it and record the visible result.



SO **Observed tosses provide evidence about the hidden coin.**

NEXT Start with exact inference on one small bag.

Part 1 asks what work we are repeating

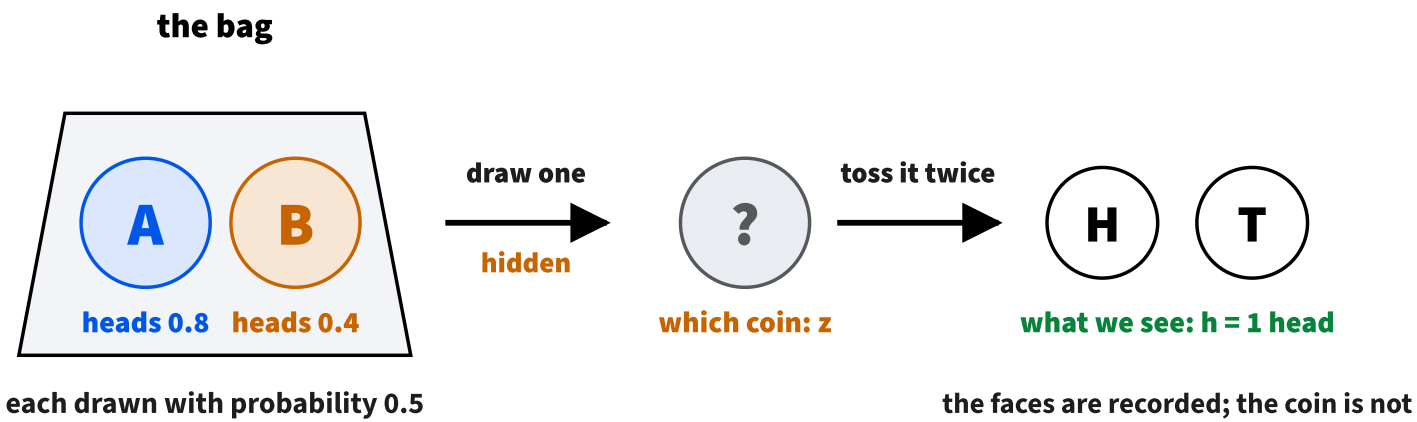
Start with exact inference on one small bag.

1 bill2 promise3 confession4 catch5 machinery6 trap7 proof8 payoff

PROBLEM To learn, we must infer the hidden part of every record, and infer it again after every update of the model.

WE DO Do that inference once by hand on the bag, then count how often it has to be repeated.

WHY That count is the work Part 2 removes.



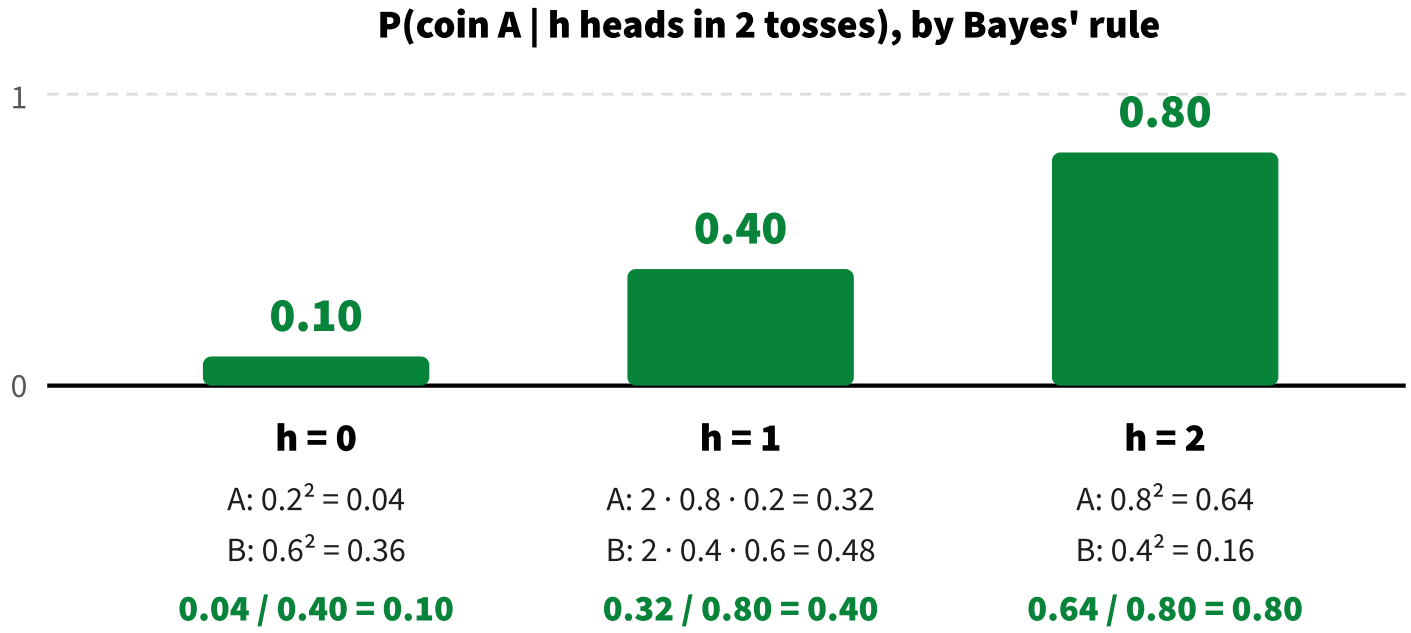
SO **First understand the calculation that the encoder will replace.**

NEXT For each possible head count, Bayes gives a probability for coin A.

Bayes' rule gives three numbers: 0.1, 0.4 and 0.8

For each possible head count, Bayes gives a probability for coin A.

$$P(A \mid h) = \frac{\frac{1}{2} \cdot p(h \mid A)}{\frac{1}{2} \cdot p(h \mid A) + \frac{1}{2} \cdot p(h \mid B)} = \frac{p(h \mid A)}{p(h \mid A) + p(h \mid B)}$$

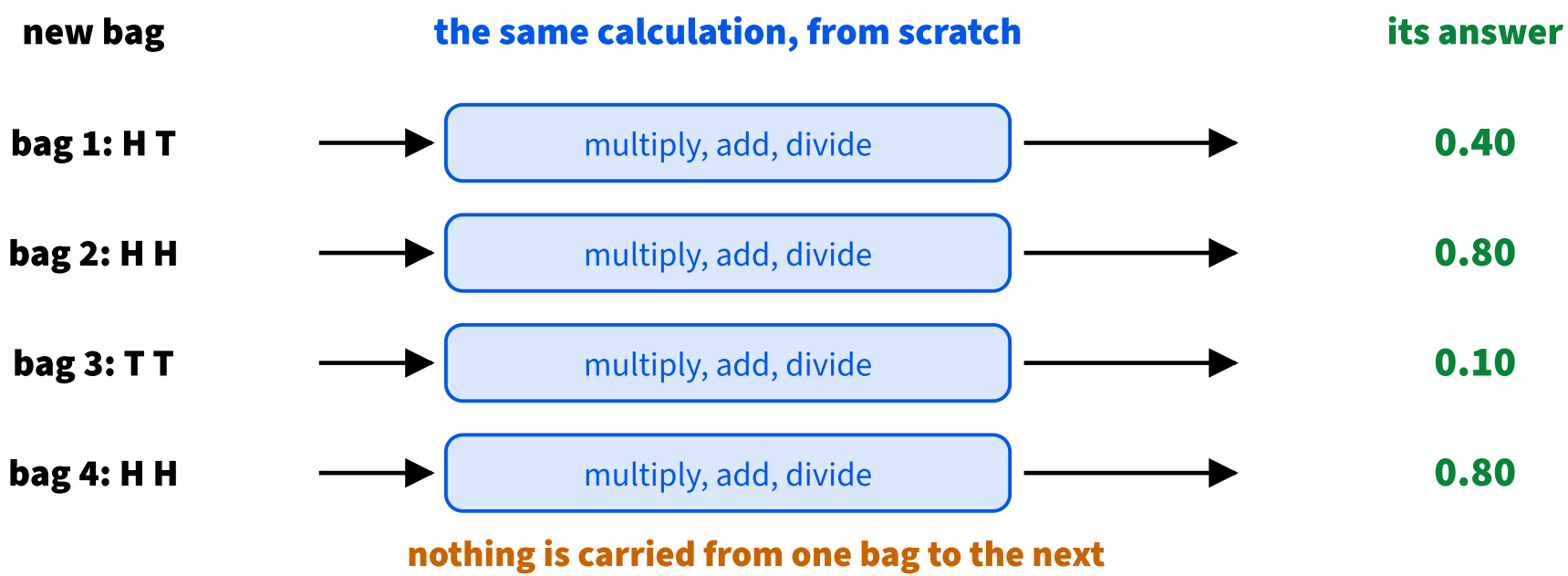


SO **The posterior changes with the observed result.**

NEXT Now another bag arrives under the same model.

The next bag starts the same calculation again

Now another bag arrives under the same model.



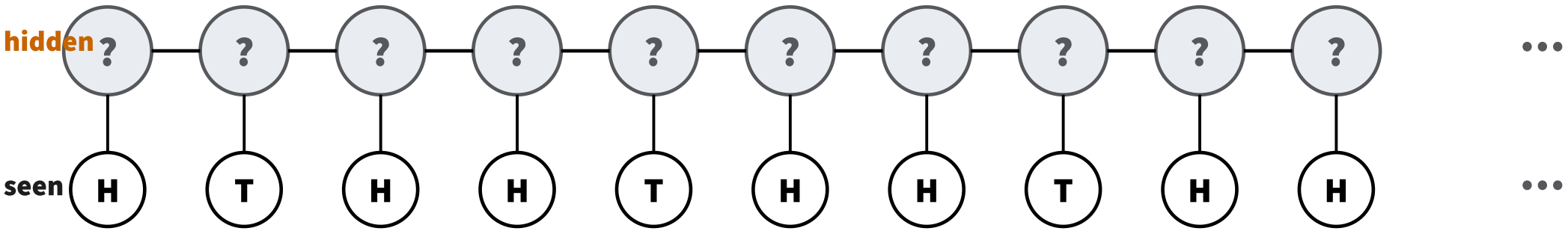
SO **A new observation still requires a new posterior calculation.**

NEXT Two hidden choices are easy to list; a large hidden configuration is not.

With twenty hidden coins there is no table to look up

Two hidden choices are easy to list; a large hidden configuration is not.

twenty coins in a row, each hidden, each showing a face



the table of hidden combinations has 2^{20} = 1 048 576 rows

so the posterior is not looked up; an algorithm computes it, for every new row of data

SO **Repeated inference becomes expensive as the hidden part grows.**

NEXT The bags share a model, so their inference problems share structure.

Pay once at training, answer every observation free

The bags share a model, so their inference problems share structure.

> Per observation: **run an optimisation**. → Once, at training: **fit a function**, then read it off.

↳ Spreading a **fixed** cost over many uses is called **amortizing** it.

When	Work
Training	Fit one reusable encoder
New bag	One encoder evaluation

SO **Pay a training cost once, then reuse a learned inference function.**

NEXT We now replace the repeated calculation with a parameterised function.

Part 2 replaces the calculation by a function

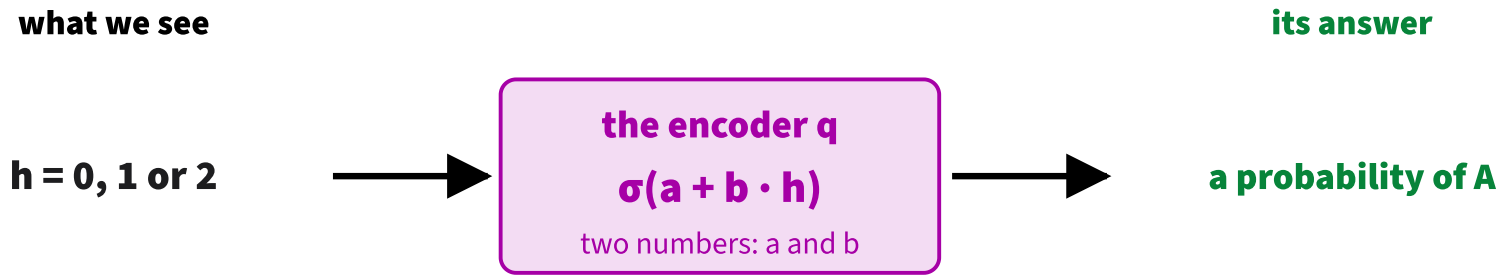
We now replace the repeated calculation with a parameterised function.

1 bill2 promise3 confession4 catch5 machinery6 trap7 proof8 payoff

PROBLEM Each observation starts its own calculation (p. 96).

WE DO Fit one function from observation to **posterior**.

WHY A fitted function answers in one pass.



SO **An encoder maps the observation to a posterior guess.**

NEXT Our input is the head count; our output is a probability for coin A.

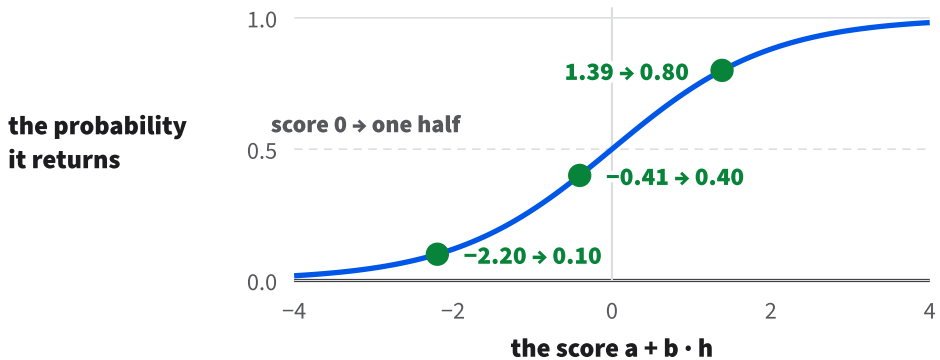
An encoder maps an observation to a distribution

Our input is the head count; our output is a probability for coin A.

Definition. An **encoder** $q_\varphi(z \mid x)$ is a function with parameters φ that takes an observation x and returns a distribution over the **hidden variable** z. Ours is

$$q_\varphi(A \mid h) = \sigma(a + b \cdot h), \qquad \varphi = (a, b).$$

In words: $\sigma(t) = 1/(1 + e^{-t})$ **turns a score into a probability; two parameters in all, shared by every observation**



SO **The output is a distribution over hidden choices, not a recovered coin label.**

NEXT The exact two-toss answers are known, so we can test a tiny encoder .

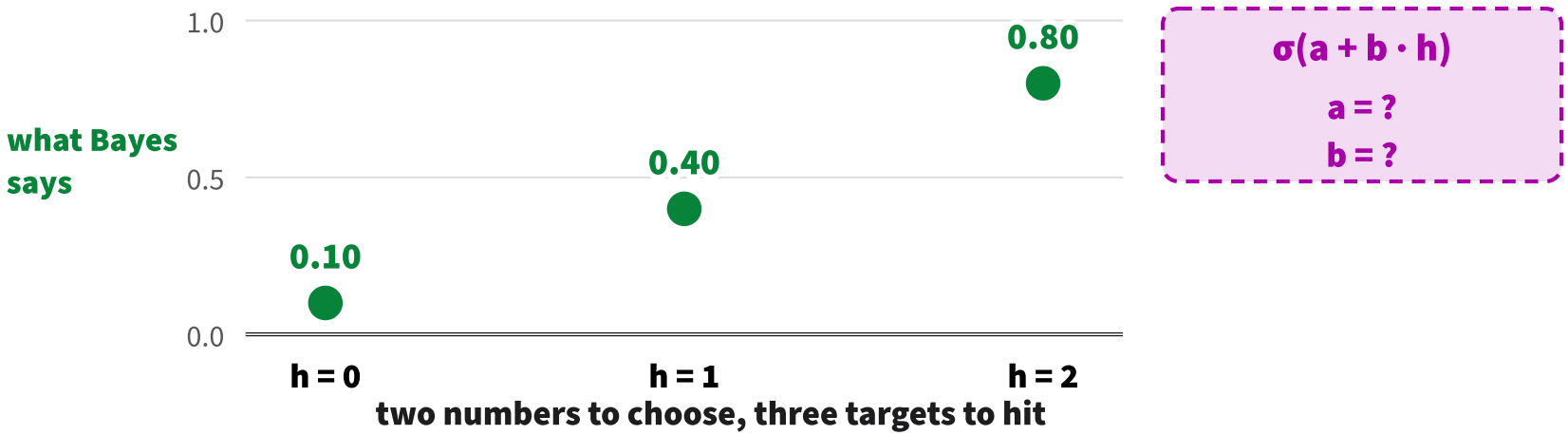
Two parameters against three exact answers: guess

The exact two-toss answers are known, so we can test a tiny **encoder**.

How close can $\sigma(a + b \cdot h)$ come to all three?

it can match all three exactly | close, but never exact | it can match at most two

Answer. All three, **exactly**, and by a property of the model rather than luck.

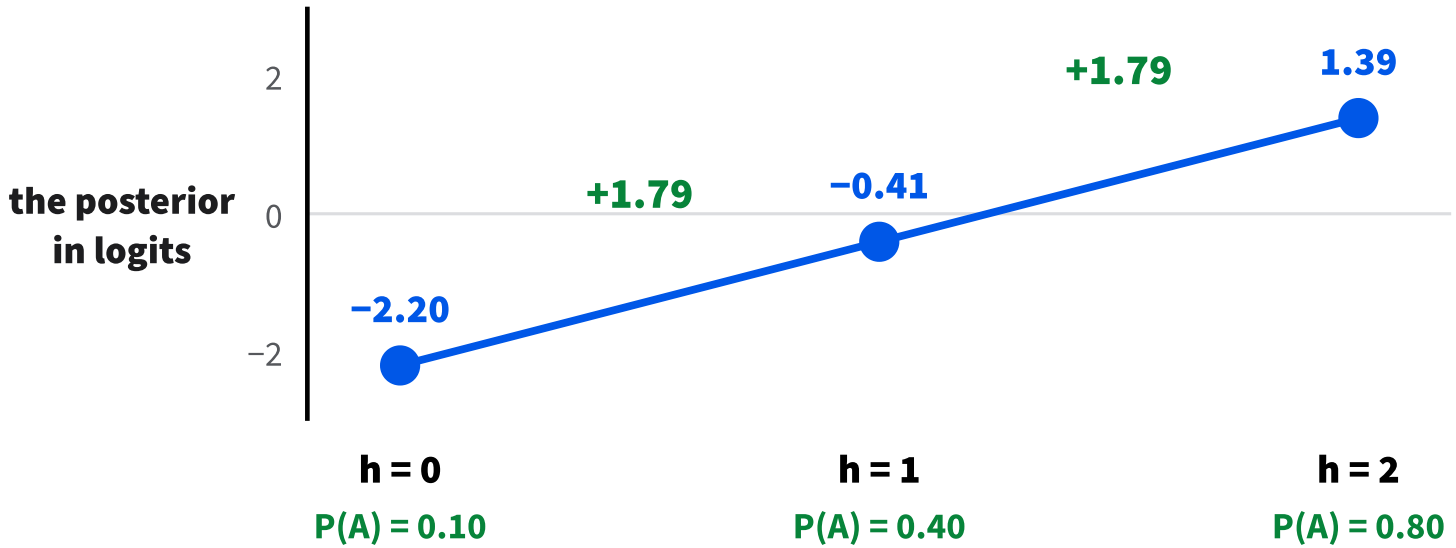


SO **A simple shared function may reproduce several separate answers.**

NEXT A probability curve becomes a straight line when expressed as log odds.

In logits the three answers lie on a straight line

A probability curve becomes a straight line when expressed as log odds.



each extra head adds the same 1.79 = log 6

so the three points lie on a straight line

a logit of 0 means the two coins are equal

SO **The logit reveals a form the encoder can express.**

NEXT Choose the intercept and slope from the coin model's log odds.

With $a = \log \frac{1}{9}$ and $b = \log 6$ the encoder is exact

Choose the intercept and slope from the coin model's log odds.

the encoder with a = log(1/9) = −2.20 and b = log 6 = 1.79

	h = 0	h = 1	h = 2
Bayes' rule	0.10	0.40	0.80
the encoder	0.10	0.40	0.80

not an approximation: the same three numbers, from two parameters

SO **For these fixed two-toss bags, two shared parameters reproduce Bayes.**

NEXT The exact fit came from the mathematical structure of this small model.

For conjugate models the exact encoder is a line

The exact fit came from the mathematical structure of this small model.

Theorem (conjugate exponential families).

$$p(z) \propto e^{\lambda \cdot t(z)}, \quad p(x \mid z) \propto e^{u(x) \cdot t(z)} \implies p(z \mid x) \propto e^{(\lambda + u(x)) \cdot t(z)}$$

In words: $t(z)$: the shape, the same in all three; λ: where the prior sits; $u(x)$: what the observation adds

↳ Our coins: $t(z) = 1[z = A]$ and $u(x) = a + b \cdot h$, so the **encoder** is affine in h and has **no gap**. Proof and the Gaussian case on **(p. 74)**.



same shape t(z) throughout; the observation adds to λ and never reshapes

SO **Special model structure can make exact inference a simple function.**

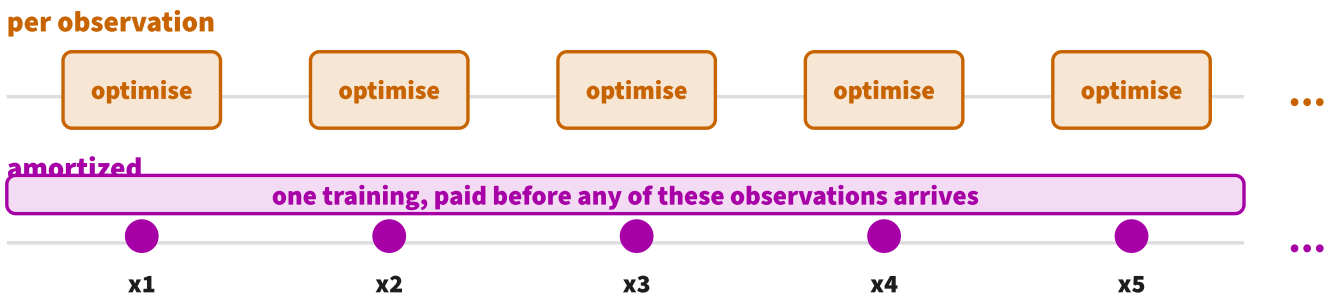
NEXT We fitted one rule instead of solving every bag independently.

Amortized inference: fit the map, then read it off

We fitted one rule instead of solving every bag independently.

Definition. **Amortized inference** replaces the per-observation optimisation $q^\star(x) = \arg\max_q \mathcal{L}(q; x)$ by one fitted function $q_\varphi(\cdot \mid x)$, trained once and then read off.

	per observation	amortized
cost of a new observation	**an optimisation**	**one forward pass**
what is stored	**nothing**	**the parameters φ**
best answer reachable	**the exact posterior**	**the best member of the family**



SO **Amortized inference shares the work of inference across observations.**

NEXT The example let us check exact answers, but real training cannot rely on them.

Part 3 trains the encoder without knowing the answer

The example let us check exact answers, but real training cannot rely on them.

1 bill2 promise3 confession4 catch5 machinery6 trap7 proof8 payoff

PROBLEM On the bag we could compute the **posterior** and copy it. In a real model, computing it per record **is** the cost we are avoiding.

WE DO Train on a score that needs no **posterior**: the **evidence lower bound**.

WHY That score is highest **exactly** at the posterior, so climbing it lands there.

the model θ	**the encoder ϕ**	**lower bound L**
$p(z)$ and $p(x \mid z)$	$q(z \mid x) = \sigma(a + b \cdot h)$	one number per observation
today: known, fixed	**today: the two numbers we fit**	**what we maximize**

in a full variational autoencoder θ and ϕ are trained together, on the same bound

we keep θ fixed so that every number today can be checked by hand

SO **Train on a computable score instead of posterior labels.**

NEXT What can we calculate without knowing the posterior?

The model is known; the sum over the hidden part is not

We can evaluate a proposed hidden choice even when summing all choices is expensive.

z: **hidden coin** A or B. x: observed toss result. $p(z)$ chooses the coin; $p(x \mid z)$ generates its toss result.

We specify both. Computing the evidence requires adding contributions from **every possible hidden choice**.

we can evaluate it at one z

p(z) and p(x | z) are formulas we wrote:
for the bag, 0.5 and the two-toss count.
Plug in a z, get a number.

cheap, and all the bound needs

we cannot add it over every z

p(x) = Σ over all z of p(z) · p(x | z),
and the posterior divides by it.
Twenty hidden coins: 1 048 576 terms.

this is the part we avoid

the model is known; it is the sum over the hidden part that is out of reach

SO **The model's joint probability is available point by point.**

NEXT Which **computable score** is tied to the **posterior** we want?

The evidence lower bound sits below $\log p(x)$, short by a divergence

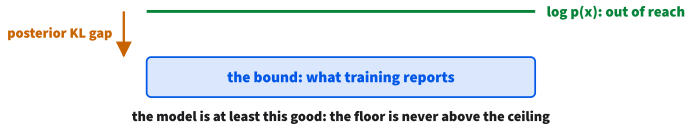
We need a computable score that rewards useful posterior guesses.

$\mathcal{L}(q) = \mathbb{E}_{z\sim q}\left[\log \frac{p(x,z)}{q(z)}\right]$ an average of one ratio: the model's own density over our own q. Both are ours to compute, so this number is too.

$$\log p(x) = \underbrace{\mathcal{L}(q)}_{\text{computable}} + \underbrace{\mathrm{KL}(q \,\|\, p(z \mid x))}_{\geq 0,\ \text{zero only at the posterior}}$$

In words: with the model fixed, a higher floor means a smaller divergence from the posterior; the encoder family may prevent an exact match

↳ Three lines, on **(p. 66)**; the same split as Lemma 1 of Lecture 5 **(Lecture 5, p. 47)**.



SO **At a fixed model, a higher $\mathcal{L}$ means a smaller posterior divergence.**

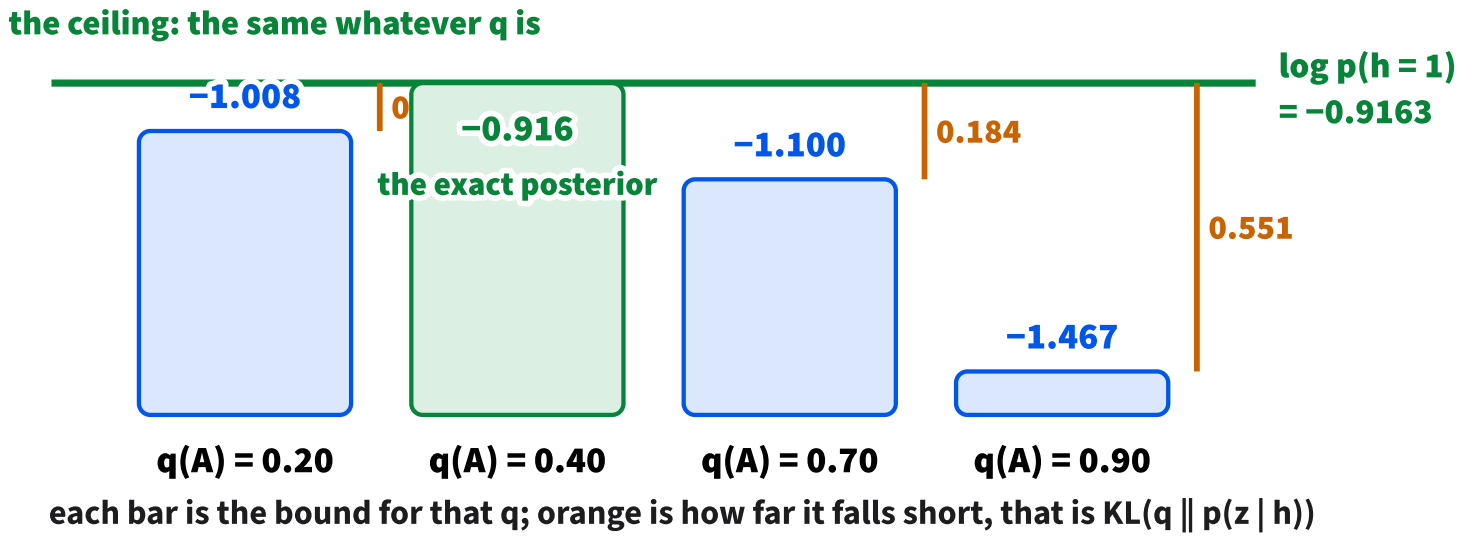
NEXT Use one observed bag to see how different guesses receive different scores.

Every guess gives a floor; only the posterior touches

Use one observed bag to see how different guesses receive different scores.

For one head in two tosses: $p(x, A) = 0.16$, $p(x, B) = 0.24$, so $p(x) = 0.40$.

For $q(A) = 0.20$, the bar is $0.20 \log(0.16/0.20) + 0.80 \log(0.24/0.80) \approx -1.008$. The orange gap is $\log(0.16 + 0.24) - \mathcal{L}(q) \approx 0.091$.



SO **Only the exact posterior makes the lower bound $\mathcal{L}$ equal the log evidence.**

NEXT One bag supplies one score; a shared encoder must learn from all bags.

One shared encoder learns from every bag

One bag supplies one score; a shared encoder must learn from all bags.

x_k is the observed result from bag k; φ denotes the **shared encoder** parameters.

$$\mathcal{L}_k(\varphi) = \mathbb{E}_{z\sim q_\varphi(\cdot|x_k)}[\log p(x_k, z) - \log q_\varphi(z \mid x_k)]$$

For two bags, add their scores. Update the **same encoder parameters** using both.

$$\mathcal{L}_{\text{total}}(\varphi) = \mathcal{L}_1(\varphi) + \mathcal{L}_2(\varphi)$$

For any data set, sum over all bags. The model is **fixed** here; only the **encoder** changes.

Bag	Its contribution
First bag	$\mathcal{L}_1(\varphi)$
Second bag	$\mathcal{L}_2(\varphi)$
Together	$\mathcal{L}_1 + \mathcal{L}_2$: update the same parameters

SO **Sum the scores and update the same encoder parameters.**

NEXT Before learning the model too, clarify exactly what the training score guarantees.

The training score is a lower bound

Before learning the model too, clarify exactly what the training score guarantees.

$\log p(x, z) - \log q_\varphi(z \mid x)$ underestimates the **log evidence**. Their difference measures how far our guess is from the **posterior**.

$$\log p_\theta(x) = \mathcal{L}(\theta, \varphi; x) + \underbrace{\mathrm{KL}(q_\varphi(z \mid x) \| p_\theta(z \mid x))}_{\text{nonnegative gap}}$$

θ controls the model; φ controls the **encoder**. With the model **fixed**, raising $\log p(x, z) - \log q_\varphi(z \mid x)$ reduces this gap.

If both change, a higher $\mathcal{L}$ alone does not guarantee higher log evidence. Proof (p. 77).

What we can compute	What it guarantees
Lower bound	No larger than log evidence
Higher bound, fixed model	Smaller posterior gap
Higher bound, changing model	No monotonic likelihood guarantee

SO **the lower bound $\mathcal{L}$ underestimates log evidence at each fixed model.**

NEXT How can we compute the same score using fit and a prior penalty?

The score rewards fit and penalizes prior divergence

The same training score can be read in a form that needs no posterior.

> Original: $\mathcal{L} = \mathbb{E}_q[\log p(x, z) - \log q(z \mid x)]$

Fit: do the coins favoured by the **encoder** make the observed toss result likely?

Penalty: how far does the **encoder** move from the original equal probabilities?

> Regrouped: $\mathcal{L} = \underbrace{\mathbb{E}_q[\log p(x \mid z)]}_{\text{fit to the observed result}} - \underbrace{\mathrm{KL}(q(z \mid x) \| p(z))}_{\text{departure from the prior}}$

This is the **same score**, regrouped using $p(x, z) = p(x \mid z)p(z)$. The **penalty** uses the **prior**; the previous slide's gap uses the **posterior**.

SO **The prior penalty and the posterior gap $\mathrm{KL}(q(z \mid x) \| p(z \mid x))$ are different quantities.**

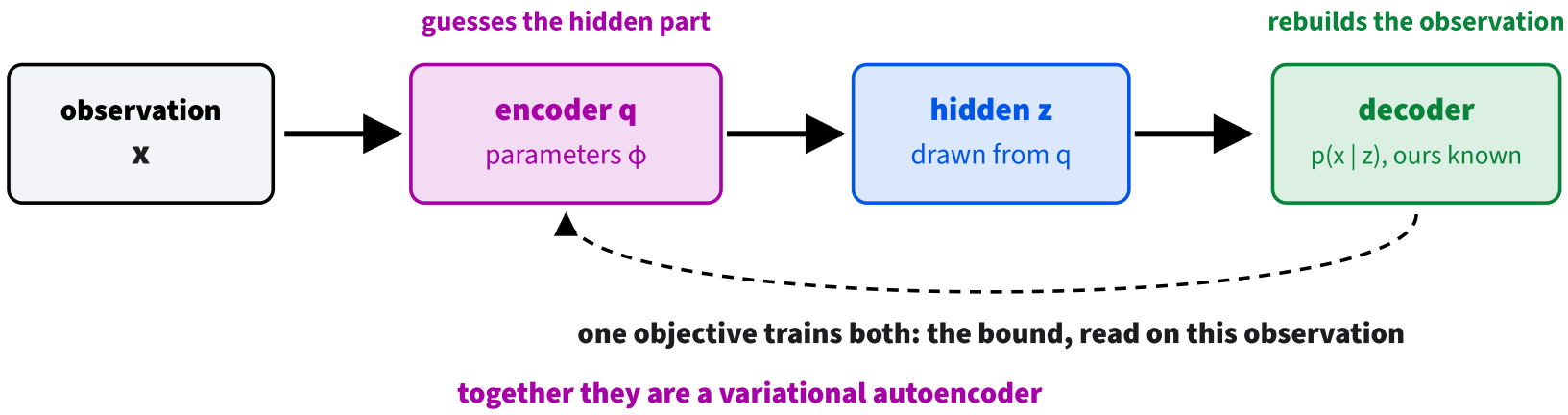
NEXT So far the coin probabilities were supplied; now let the generative model learn too.

Two networks learn the guesses and the model

So far the coin probabilities were supplied; now let the generative model learn too.

Encoder: reads a bag's toss result x, then outputs a guess about its **hidden coin** z.

Decoder: takes a hidden choice z, then specifies probabilities for possible observations x.



Train both with the same bound. With **neural networks**, this is a **variational autoencoder**. The **encoder** outputs a distribution, not a known hidden answer.

SO **Train the encoder and decoder together using the lower bound.**

NEXT We have met the two tasks of inferring hidden choices and updating the model before.

The same two tasks return from earlier lectures

We have met the two tasks of inferring hidden choices and updating the model before.

Task	Expectation–maximization	Variational autoencoder
Guess the hidden coin	**Compute each bag's posterior**	**One encoder predicts each distribution**
Improve the model	**Update the coin probabilities**	**Update the decoder parameters**

Both use the **evidence lower bound**. A **shared encoder** replaces separate **posterior** calculations.

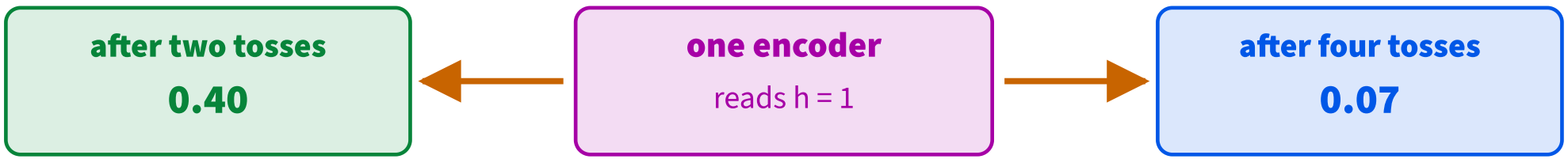
The connection is conceptual: joint **gradient** updates **need not** perform exact E-steps or M-steps, or preserve their likelihood improvement guarantee.

SO **A variational autoencoder shares the E-step's work through an encoder.**

NEXT The two-toss encoder fitted exactly; now test whether that success survives a change.

Why can a shared encoder give the wrong answer?

The two-toss **encoder** fitted **exactly**; now test whether that success survives a change.



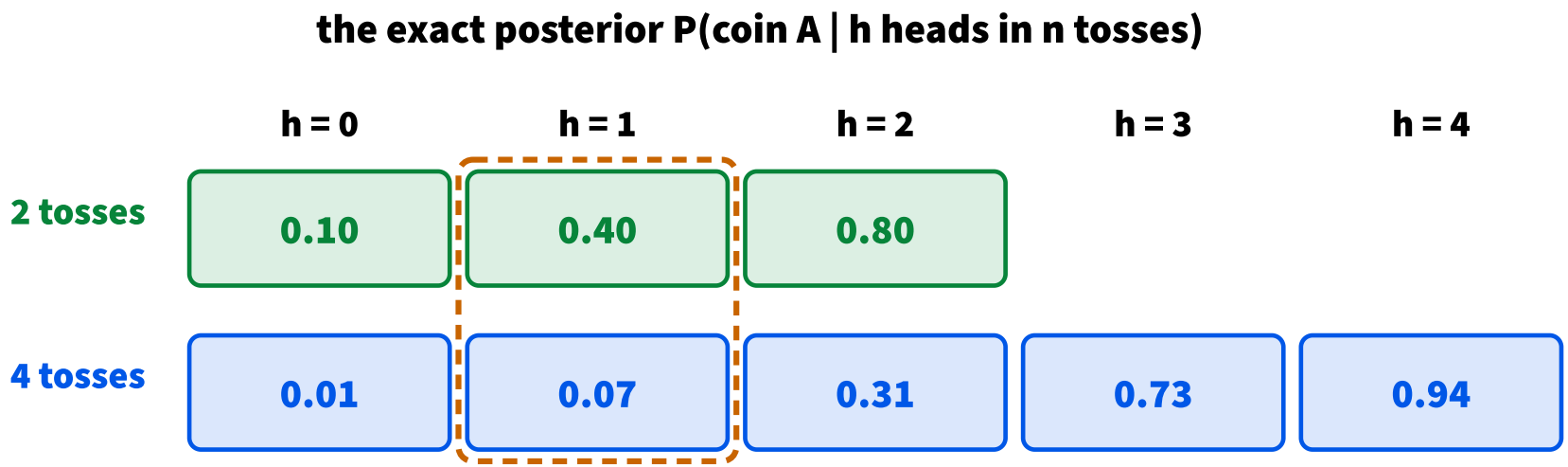
the same count, two different right answers, and one number to give

SO **Sharing a function can impose restrictions on its answers.**

NEXT Compare one head in two tosses with one head in four tosses.

One head means different things in different bags

Compare one head in two tosses with one head in four tosses.



at h = 1 the same count means 0.40 after two tosses and 0.07 after four

one head out of two is evidence for A; one head out of four is evidence against it

SO **The same head count can require different posterior answers.**

NEXT Our encoder receives only the head count, so those different bags look identical.

Identical inputs force identical encoder outputs

Our **encoder** receives only the head count, so those different bags look identical.

the best encoder that reads h alone: σ(−2.3297 + 1.3028 · h)

	h = 0	h = 1	h = 2	h = 3	h = 4
exact, 2 tosses	0.10	0.40	0.80		
exact, 4 tosses	0.01	0.07	0.31	0.73	0.94
the encoder	0.09	0.26	0.57	0.83	0.95

one answer per count: 0.26 sits between 0.40 and 0.07, and fits neither

SO **No function of heads alone can give both correct answers.**

NEXT Is the limitation in each separate guess, or in the shared function?

Separate guesses can succeed; sharing can fail

Observed bag	Separate best guess	Encoder reads only heads
1 head / 2 tosses	$q(A) = 0.40$	**Same input**: $h = 1$
1 head / 4 tosses	$q(A) \approx 0.069$	**Same output**: **cannot match both**

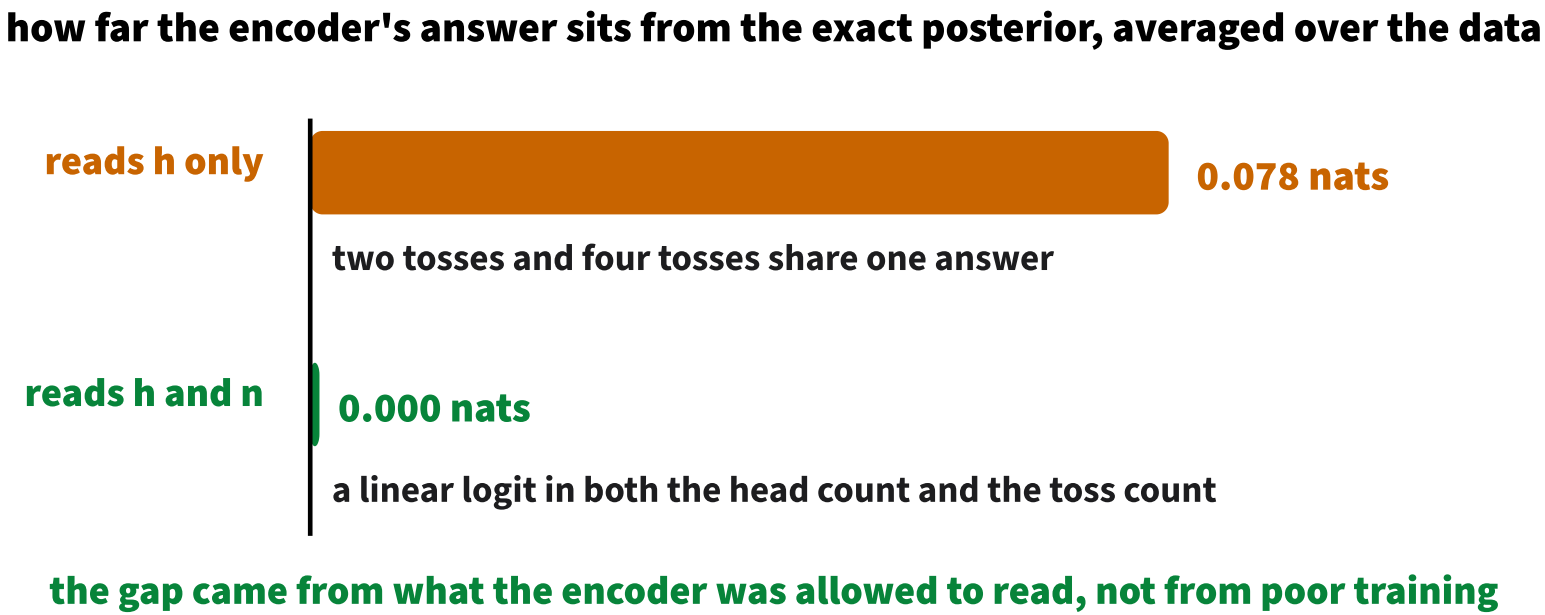
Distinguish the best separate guesses from the best shared encoder.

Approximation: loss in the best **separate guesses**.
Amortization: extra loss from the **shared encoder**.

SO **Approximation restricts separate guesses; amortization adds the cost of sharing.**
NEXT Can supplying the missing toss count remove this conflict?

Reading both counts removes this input conflict

The conflict came from an input that omitted the number of tosses.

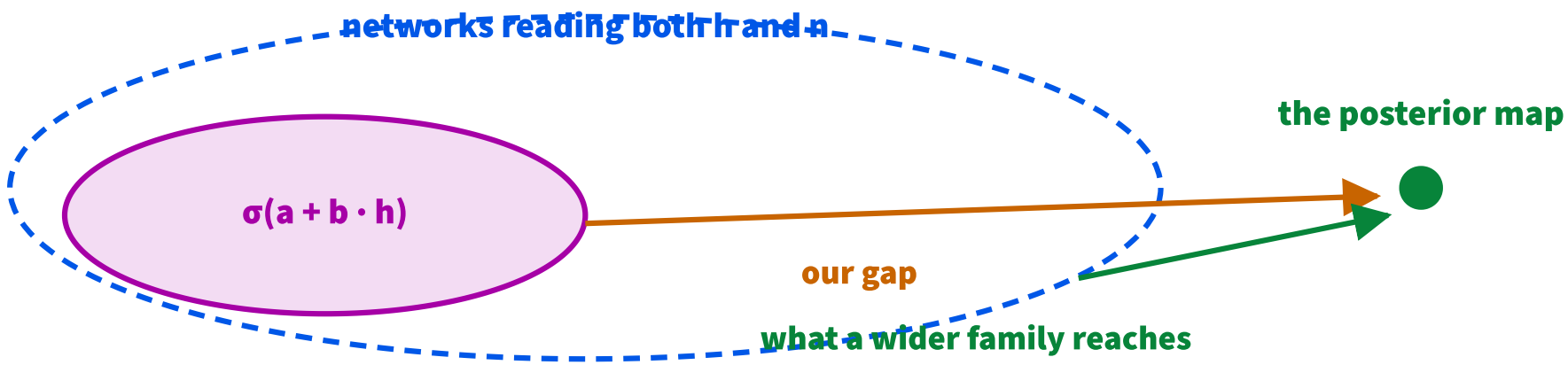


SO **Reading both counts allows this model's exact posterior rule.**

NEXT After supplying sufficient inputs, ask whether the encoder can express the needed rule.

More capacity helps only after inputs are sufficient

After supplying sufficient inputs, ask whether the **encoder** can express the needed rule.



SO **Expressiveness gives possibilities, not a guarantee that training finds them.**

NEXT Different restrictions call for different changes to the inference procedure.

Choose a remedy that matches the restriction

Different restrictions call for different changes to the inference procedure.

three ways to make the gap smaller

give it more input

let the encoder read
everything the posterior
depends on

give it a richer family

more parameters,
more layers,
a flexible output

refine per observation

start from the encoder,
then take a few steps
on this observation

the third keeps the speed of the first pass and buys back some accuracy

SO **Add inputs, enlarge the family or refine a shared guess per observation.**

NEXT We can train shared posterior guesses and explain why they may be imperfect.

We can train guesses without posterior labels

We can train shared **posterior** guesses and explain why they may be imperfect.

What we have	Question still open
Shared guesses	How do we differentiate sampled scores?
Computable score	Will training use the hidden variable?
Two gaps	Can we check a complete fitted example?

SO **Next we need gradients when the hidden variable is a number.**

NEXT Keep the coin story, but infer its unknown head probability rather than its identity.

What if we do not know the coin's head probability?

Keep the coin story, but infer its unknown head probability rather than its identity.

What we observe	What remains hidden
One head in two tosses	The coin's head probability z

> z could be $0.2,\ 0.5,\ 0.8,\ \ldots$ — **any value between zero and one**.

This is a **continuous hidden variable**. Two tosses cannot tell us its exact value.

SO **A continuous hidden variable has a range of possible values.**

NEXT What simple continuous model will let us practise this inference?

Discrete samples can be trained by their scores

A discrete coin choice cannot be moved smoothly like a continuous candidate.

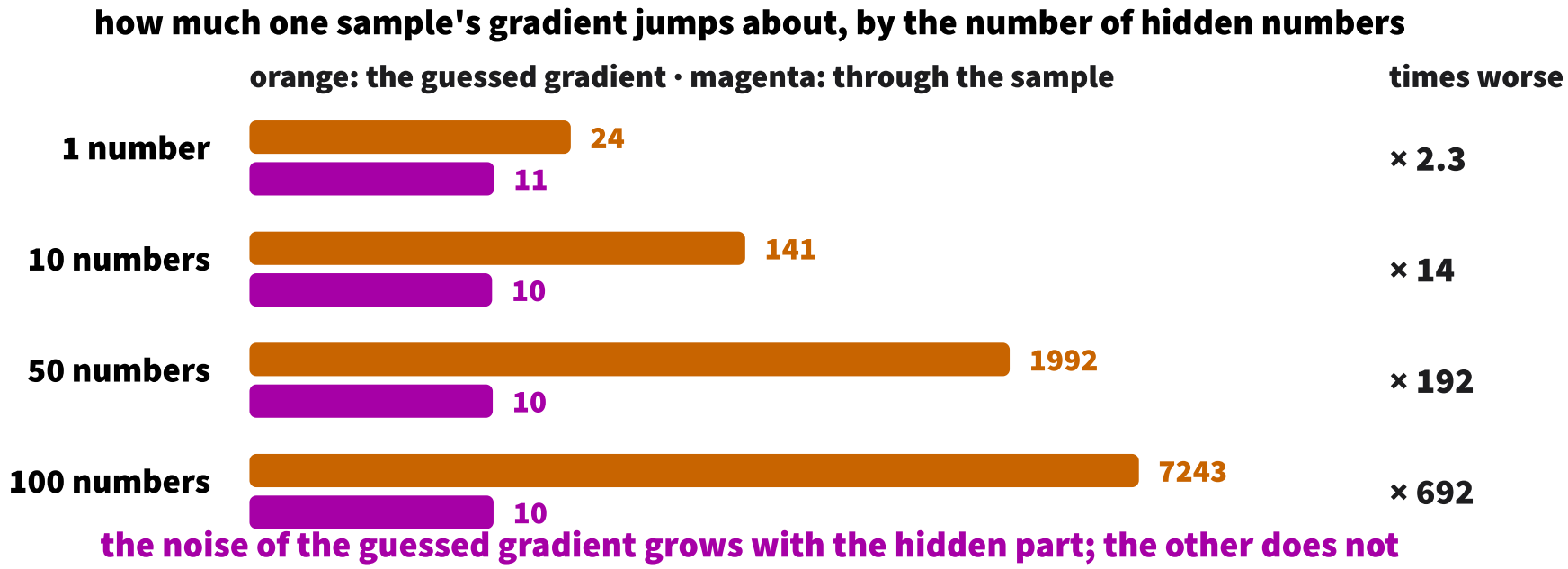
Continuous hidden number	Discrete coin identity
Move a candidate slightly	Cannot move smoothly from A to B
Differentiate through its movement	Differentiate its sampling probability

SO **Score-function gradients instead adjust the probabilities of sampled choices.**

NEXT Two correct gradient estimators can differ in how much their samples fluctuate.

Correct averages can still give noisy updates

Two correct **gradient** estimators can differ in how much their samples fluctuate.

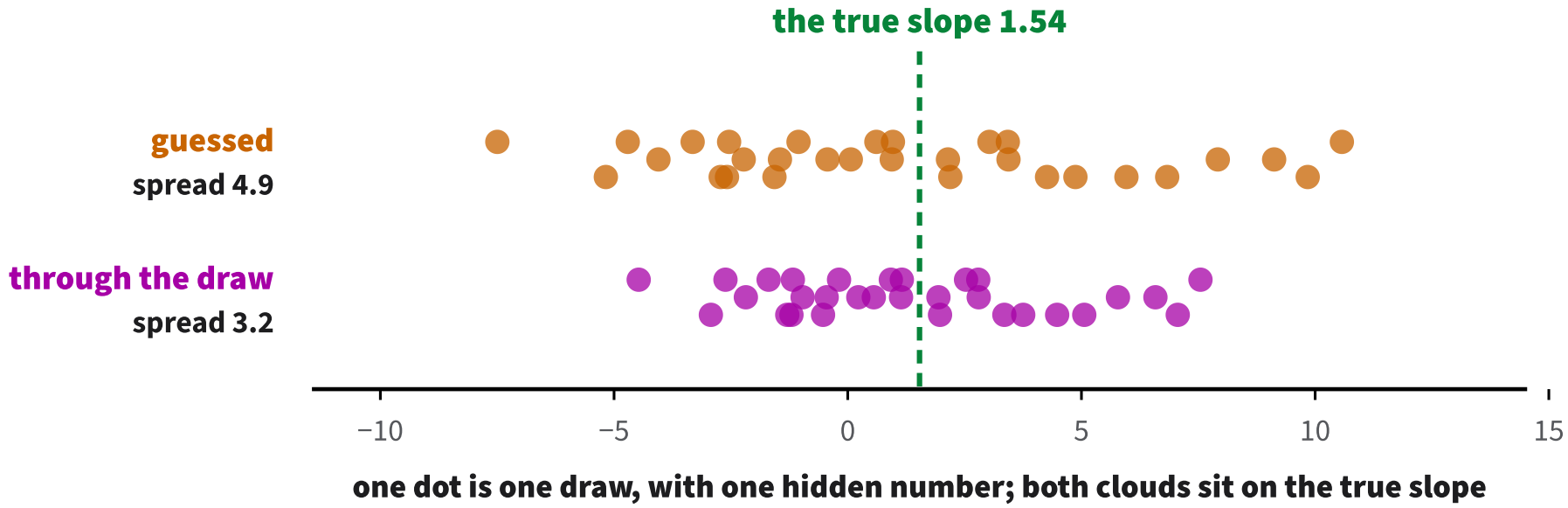


SO **Variance affects how reliably a few samples estimate the gradient.**

NEXT A noisy update is useful only if its average estimates the desired gradient.

Unbiased gradients need not reach the best solution

A noisy update is useful only if its average estimates the desired **gradient**.

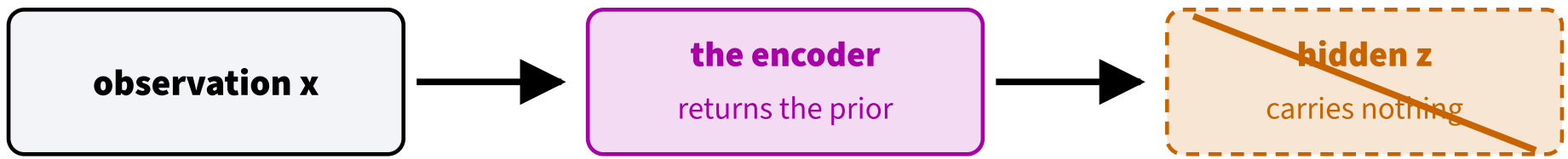


SO **Unbiasedness alone guarantees neither convergence nor a global optimum.**

NEXT Can $\log p(x, z) - \log q_\varphi(z \mid x)$ improve while the **hidden variable** becomes uninformative?

A good training score may leave the hidden part unused

We can train the **encoder**, but a good score may still leave its **hidden variable** unused.



the bound went up, and the model learned nothing about the hidden part

SO **Identical coins show what uninformative inference looks like.**

NEXT Make A and B equally likely to produce every observed toss result.

Identical coins cannot be distinguished by tosses

Make A and B equally likely to produce every observed toss result.

P(coin A | h), when both coins show heads with probability 0.6

	h = 0	h = 1	h = 2
two coins, as before	0.10	0.40	0.80
two equal coins	0.50	0.50	0.50

the observation says nothing about the coin, so the posterior is the prior

and the best encoder is the flat one: b = 0.00

SO **The observations cannot change the prior odds between identical coins.**

NEXT Name the state in which observations stop changing the encoder 's distribution.

Collapse means the guesses stop using the observation

Name the state in which observations stop changing the **encoder**'s distribution.

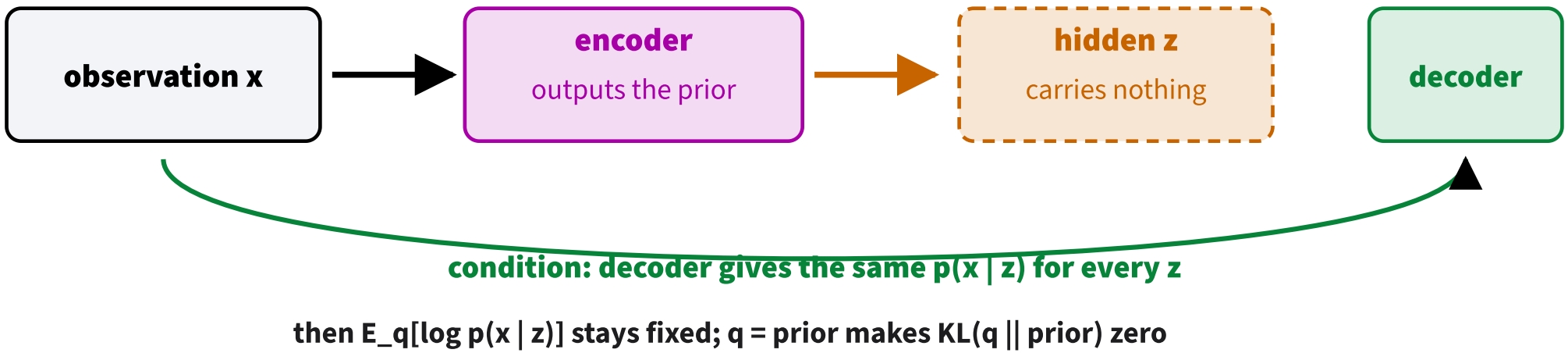
Observation	Identical-coin posterior
No heads	Half A, half B
One head	Half A, half B
Two heads	Half A, half B

SO **Posterior collapse means the encoder returns approximately the prior.**

NEXT Return to fit minus prior penalty to explain why ignoring the latent can be attractive.

Ignoring the hidden variable can reduce the penalty

Return to fit minus **prior** **penalty** to explain why ignoring the latent can be attractive.



SO **If using the latent cannot improve fit, its prior penalty is avoidable cost.**

NEXT To retain useful latent information, change the incentive or the decoder's dependence on it.

Make information useful before penalizing it strongly

To retain useful latent information, change the incentive or the **decoder**'s dependence on it.

Change	Intended effect
Start with a weak prior penalty	Let latent information become useful
Allow some information before penalising more	Reduce pressure toward an empty latent
Restrict the decoder	Make latent information help explain data

SO **Remedies alter a tradeoff; none is a universal cure.**

NEXT Put the family, score and gradient into one small model we can check.

Now train a whole encoder in a checkable model

Put the family, score and **gradient** into one small model we can check.



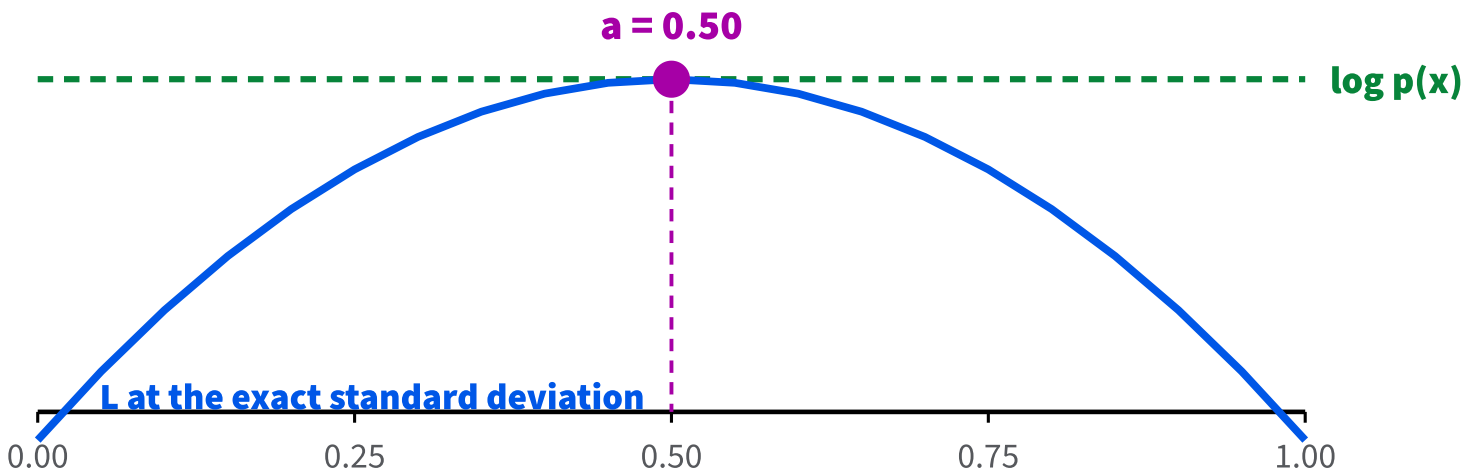
every number below is one you can reproduce by hand

SO **A complete worked example lets us verify the encoder against Bayes.**

NEXT Our Gaussian encoder needs a rule for its mean and a positive spread.

The lower bound identifies better encoder parameters

Before updating parameters, see how their choices change $\log p(x, z) - \log q_\varphi(z \mid x)$.



SO **the lower bound $\mathcal{L}$ reaches the evidence only at the exact posterior.**

NEXT Compute $\log p(x, z) - \log q_\varphi(z \mid x)$ slopes at the current parameters, then take one small step.

Each new tool answered a specific obstacle

Review each obstacle and the tool introduced to answer it.

Obstacle	Tool
Repeated inference	Shared encoder
No posterior labels	Evidence lower bound
Restricted guesses	Approximation and amortization gaps
Samples in the objective	Reparameterization
Unused hidden variable	Check for collapse

SO **The tools belong to one inference story, rather than unrelated formulas.**

NEXT Follow the path from repeated Bayes calculations to one trained inference function.